



P.I.L.

Public Information Limited, LLC.

Data Analysis with Python

Version 1.0.0-v0.0.0

Table of Contents

Module 1: Data Visualization

Lesson 1.1: Importance of Data Visualization in Data Analysis

Lesson 1.2: Interactive Visualizations Using Plotly

Lesson 1.3: Static Plots Using Matplotlib

Lesson 1.4: Advanced Visualization Techniques with Seaborn

Module 2: Data Manipulation

Lesson 2.1: Fundamentals of Data Manipulation using NumPy

Lesson 2.2: Data Manipulation and Analysis using Pandas

Lesson 2.3: Features of Polars for Efficient Data Manipulation

Module 3: Statistical Analysis

Lesson 3.1: Principles of Statistical Analysis

Lesson 3.2: Regression Analysis and Hypothesis Testing with Statsmodels

Lesson 3.3: Advanced Statistical Analysis Techniques with Pingouin

Lesson 3.4: Statistical Computations and Hypothesis Testing with SciPy

Module 1: Data Visualization

Exercise 1A - Create an interactive Plotly visualization displaying trends in a dataset.

Exercise 1B - Generate a static Matplotlib plot illustrating the distribution of a numerical variable.

Exercise 1C - Use Seaborn to visualize the relationship between multiple variables in a dataset.

Exercise 1D - Combine Plotly, Matplotlib, and Seaborn to create a comprehensive dashboard showcasing various aspects of a dataset.

Module 2: Data Manipulation

Exercise 2A - Create NumPy arrays and perform basic array manipulations such as reshaping and indexing.

Exercise 2B - Use Pandas to load, clean, and analyze a dataset, performing tasks such as data filtering and aggregation.

Exercise 2C - Explore the features of the Polars library by performing data manipulation tasks and comparing their efficiency with NumPy and Pandas.

Module 3: Statistical Analysis

Exercise 3A - Perform exploratory data analysis on a given dataset, including calculating descriptive statistics and visualizing distributions.

Exercise 3B - Use Statsmodels to perform linear regression analysis and hypothesis testing on a dataset.

Exercise 3C - Explore advanced statistical analysis techniques using Pingouin, including conducting correlation analysis and non-parametric tests.

Exercise 3D - Conduct hypothesis tests and statistical computations using SciPy.

1

2

7

13

19

24

25

31

36

41

42

47

53

58

64

65

66

67

68

70

71

72

73

75

76

78

80

82

Module 1: Data Visualization



Learning Objectives



Upon completion of this module, students should be able to:

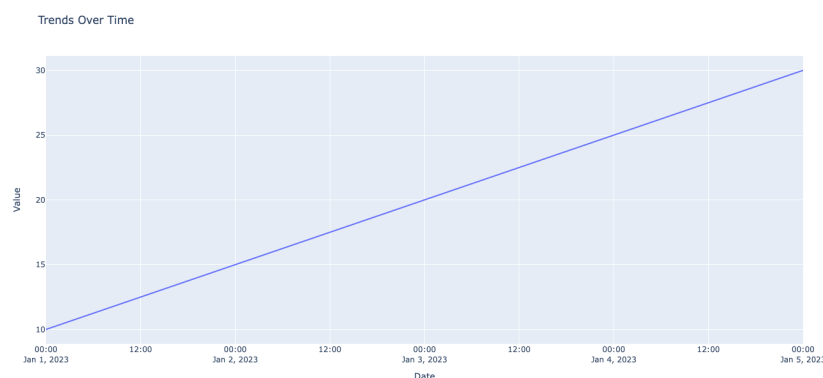
- Understand the importance of data visualization in data analysis.
- Create interactive visualizations using Plotly.
- Master the creation of static plots using Matplotlib.
- Explore advanced visualization techniques with Seaborn

Lesson 1.1: Importance of Data Visualization in Data Analysis



Data visualization plays a crucial role in the field of data analysis, serving as a powerful tool for understanding, interpreting, and communicating insights derived from data.

In this lesson, we'll explore the significance of data visualization and its impact on the data analysis process.



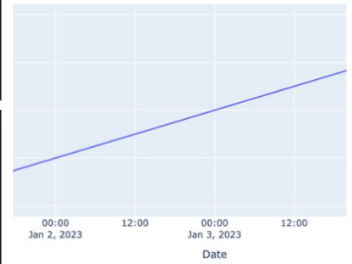
- Data visualization is a crucial component of data analysis, aiding in the interpretation and communication of insights derived from data.
- Effective data visualization enhances understanding by presenting complex information in a clear and intuitive manner.
- Visualizations can reveal patterns, trends, and relationships in data that may not be apparent from raw numbers alone.
- Different visualization techniques serve various purposes, from exploratory analysis to presentation of findings to stakeholders.
- Interactive visualizations allow users to explore data dynamically and gain deeper insights through exploration and interaction.

Understanding Data Visualization

- Data visualization refers to the graphical representation of data using charts, graphs, maps, and other visual elements.
- It involves transforming raw data into visual formats that are easy to interpret and understand.
- By visualizing data, analysts can identify patterns, trends, and relationships that may not be apparent from raw data alone.

```
1 date_column,value_column
2 2023-01-01,10
3 2023-01-02,15
4 2023-01-03,20
5 2023-01-04,25
6 2023-01-05,30
```

```
1 import pandas as pd
2 import plotly.express as px
3
4 # Load your dataset
5 df = pd.read_csv('module_1_lesson_1_sampledata.csv')
6
7 # Create the Plotly visualization
8 fig = px.line(df, x='date_column', y='value_column', title='Trends')
9
10 # Customize the visualization
11 fig.update_layout(xaxis_title='Date', yaxis_title='Value')
12
13 # Show the visualization
14 fig.show()
```



- Data visualization encompasses the graphical representation of data through charts, graphs, maps, and other visual elements.
- Its purpose is to convert raw data into visually comprehensible formats that facilitate interpretation and understanding.
- Through data visualization, analysts can uncover patterns, trends, and correlations that might be obscured within raw data.

Principles of Data Visualization

Effective data visualization is guided by a set of principles and best practices to ensure clarity, accuracy, and relevance, including:

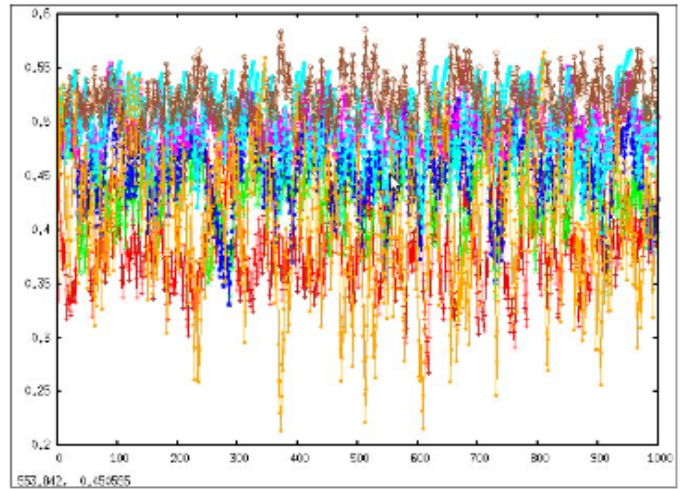
- **Clarity and Simplicity:** Visualizations should be clear and easy to understand, avoiding unnecessary complexity.
- **Accuracy and Precision:** Ensure that visualizations accurately represent the underlying data without distorting information.
- **Relevance:** Focus on presenting relevant information that addresses specific objectives or questions.
- **Interactivity:** Incorporate interactive features to allow users to explore data dynamically.
- **Accessibility:** Ensure that visualizations are accessible to all users, including those with disabilities.



-
- **Clarity and Simplicity:** Visualizations should prioritize clarity and simplicity, avoiding unnecessary complexity that may hinder understanding.
 - **Accuracy and Precision:** Ensure that visualizations accurately depict the underlying data without introducing distortions or misrepresentations.
 - **Relevance:** Focus on presenting information that is directly relevant to the intended audience and addresses specific objectives or questions.
 - **Interactivity:** Incorporate interactive elements into visualizations to allow users to explore data dynamically and gain deeper insights.
 - **Accessibility:** Ensure that visualizations are accessible to all users, including those with disabilities, by following accessibility guidelines and standards.

Benefits of Data Visualization

- **Improved Understanding**
- **Insight Discovery**
- **Communication and Collaboration**
- **Decision Making**
- **Identification of Data Quality Issues**
- **Presentation and Reporting**



- **Improved Understanding:** Visual representations of data enhance comprehension by simplifying complex information and making it more accessible.
- **Insight Discovery:** Data visualizations aid in the discovery of patterns, trends, and relationships within datasets, leading to valuable insights.
- **Communication and Collaboration:** Visualizations serve as powerful communication tools, enabling effective sharing of findings with stakeholders and fostering collaboration.
- **Decision Making:** Data visualizations empower decision-makers to make informed and data-driven decisions by presenting insights in a clear and actionable format.
- **Identification of Data Quality Issues:** Visualizations help in identifying data quality issues such as outliers and inconsistencies, facilitating data cleaning and preprocessing efforts.
- **Presentation and Reporting:** Visualizations enhance the effectiveness of presentations and reports by providing visually engaging representations of data that are easier to understand and interpret.

Conclusion

In conclusion, data visualization is a critical aspect of data analysis, enabling analysts to gain insights, communicate findings, and make informed decisions.

By following principles of effective data visualization and leveraging its benefits, analysts can unlock the full potential of data to drive business outcomes and innovation.



Hands-On Exercise: For hands-on experience, review and complete *Exercise 1A* in the exercise guide for this course.

Additional Resources

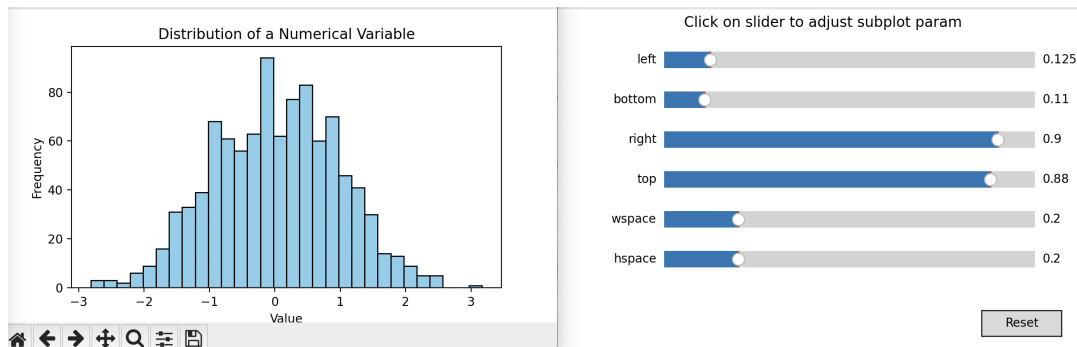
- "The Visual Display of Quantitative Information" by Edward R. Tufte
- "Storytelling with Data: A Data Visualization Guide for Business Professionals" by Cole Nussbaumer Knaflic
- Data Visualization Society: <https://www.datavisualizationsociety.com/>

Lesson 1.2: Interactive Visualizations Using Plotly



Plotly is a powerful Python library for creating interactive and visually appealing data visualizations.

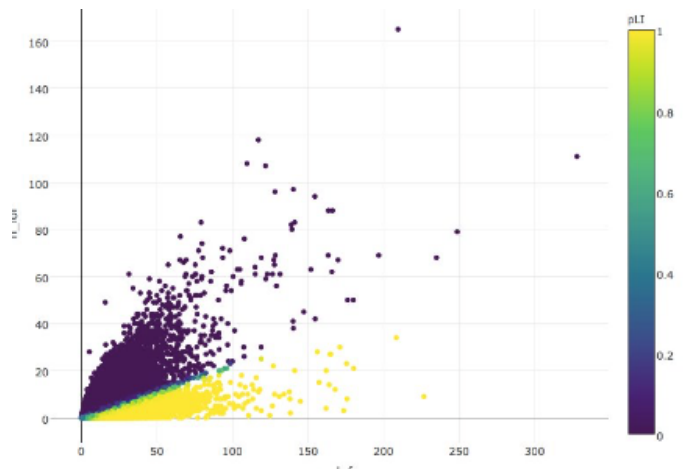
In this lesson, we'll explore the capabilities of Plotly and learn how to create interactive visualizations to enhance data exploration and analysis.



- Plotly is a powerful Python library renowned for its ability to generate interactive and visually compelling data visualizations.
- In this lesson, we delve into Plotly's extensive capabilities, discovering how to leverage its features to create interactive visualizations that enrich data exploration and analysis.

Introduction to Plotly

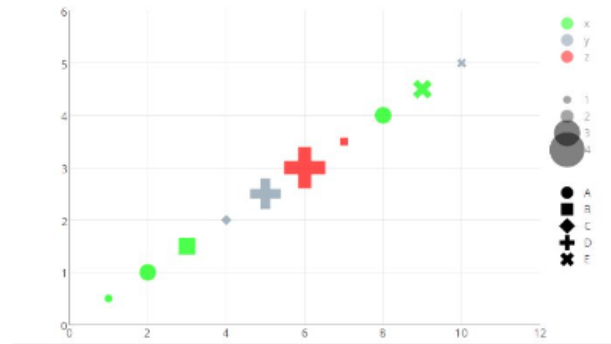
- Plotly is an open-source library that allows users to create interactive visualizations in Python.
- It provides a wide range of chart types and customization options, making it suitable for various data visualization tasks.
- Key features of Plotly include support for static and interactive plots, compatibility with Jupyter notebooks, and ability to export plots in various formats.



- Plotly offers a vast array of chart types and customization options, catering to diverse data visualization needs.
- Its robust support for interactivity includes hover effects, tooltips, zooming capabilities, and animations, enhancing user engagement with data.
- Plotly seamlessly integrates with Jupyter notebooks, providing an intuitive environment for interactive visualization development and experimentation.
- Additionally, Plotly facilitates the export of visualizations in various formats, ensuring compatibility with different platforms and applications.
- Plotly visualizations foster accessibility and collaboration by enabling easy sharing and embedding in web applications, online platforms, and reports.
- Plotly's continuous evolution and commitment to innovation drive exploration and experimentation in the realm of data visualization.

Basic Plotting with Plotly

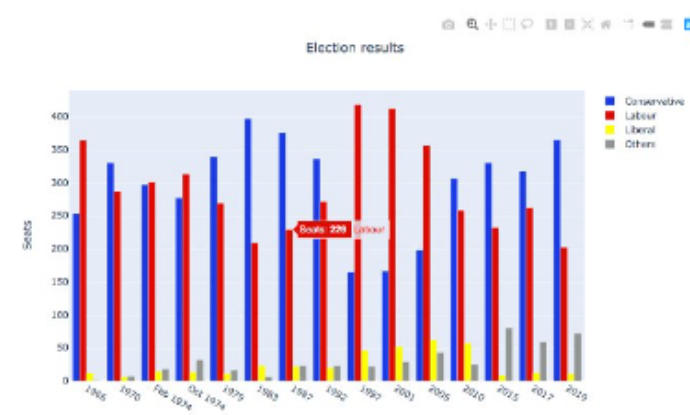
- Plotly allows users to create a variety of basic plots, including line plots, scatter plots, and bar charts.
- These plots can be generated using simple Python code, making it easy to create visualizations from data stored in Python data structures such as lists or NumPy arrays.
- Basic plotting functions in Plotly provide options for customizing the appearance of plots, including specifying colors, markers, and axis labels.



- Plotly's basic plotting capabilities extend beyond traditional chart types like line plots, scatter plots, and bar charts to include more specialized visualizations such as bubble charts, box plots, and histograms.
- In addition to Python data structures like lists or NumPy arrays, Plotly seamlessly integrates with Pandas DataFrames, allowing users to create visualizations directly from DataFrame objects.
- Basic plotting functions in Plotly offer advanced customization options such as adding trendlines, error bars, and annotations to enhance the clarity and insights of visualizations. with Pandas DataFrames, and advanced customization options beyond what was initially provided.

Enhancing Interactivity

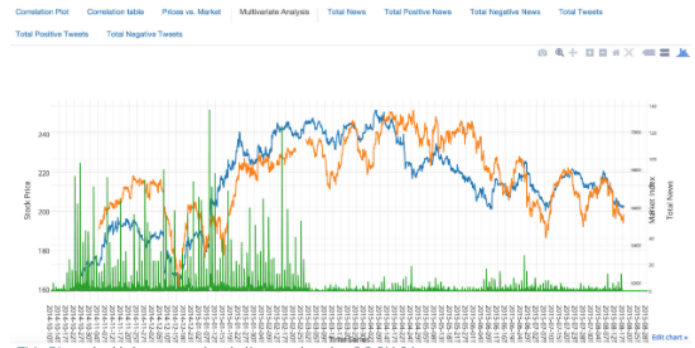
- One of the key advantages of Plotly is its support for interactivity, which allows users to interact with plots and explore data dynamically.
- Plotly visualizations can include hover effects, tooltips, and interactive controls such as sliders and dropdown menus.
- These features enhance the usability of visualizations by providing additional information and enabling users to adjust parameters and explore different aspects of the data



- Plotly excels in its support for interactivity, empowering users to interact with plots dynamically and explore data in-depth.
- Beyond standard interactions, Plotly visualizations can incorporate advanced features like hover effects, tooltips, and interactive controls such as sliders and dropdown menus.
- These interactive elements enrich the user experience by providing supplementary information and enabling users to manipulate parameters, facilitating comprehensive data exploration and analysis.

Creating Complex Visualizations

- In addition to basic chart types, Plotly supports the creation of more complex visualizations, including heatmaps, choropleth maps, and 3D plots.
- Heatmaps are useful for visualizing matrix data, while choropleth maps are used to display geographical data on maps.
- Plotly also allows users to create 3D plots for visualizing multidimensional data, such as surface plots and scatter plots in three-dimensional space.



- Plotly extends its capabilities beyond basic chart types to support the creation of more intricate visualizations, encompassing heatmaps, choropleth maps, and 3D plots.
- Heatmaps serve as effective tools for visualizing matrix data, providing a comprehensive overview of patterns and relationships within the data.
- Choropleth maps are particularly useful for displaying geographical data on maps, allowing users to visualize spatial distributions and variations across regions.
- Plotly's 3D plotting functionality enables users to visualize multidimensional data in three-dimensional space, facilitating the exploration of complex relationships and structures.

Conclusion

In conclusion, Plotly is a versatile library for creating interactive visualizations in Python.

By leveraging its features and capabilities, analysts can create engaging and informative visualizations to explore and communicate insights from data effectively.



Hands-On Exercise: For hands-on experience, review and complete *Exercise 1B* in the exercise guide for this course.

Additional Resources

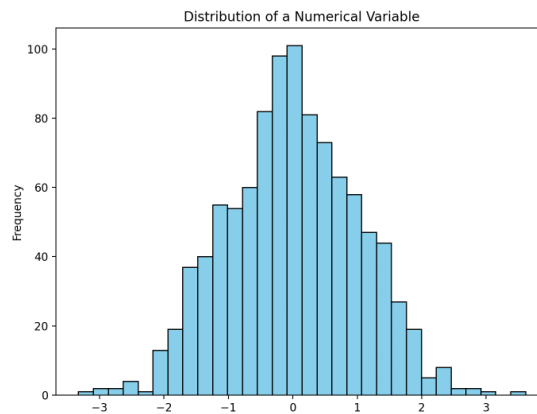
- Plotly Documentation: <https://plotly.com/python/>
- Plotly Graphing Libraries: <https://plotly.com/graphing-libraries/>

Lesson 1.3: Static Plots Using Matplotlib



Matplotlib is a popular Python library for creating static, publication-quality plots.

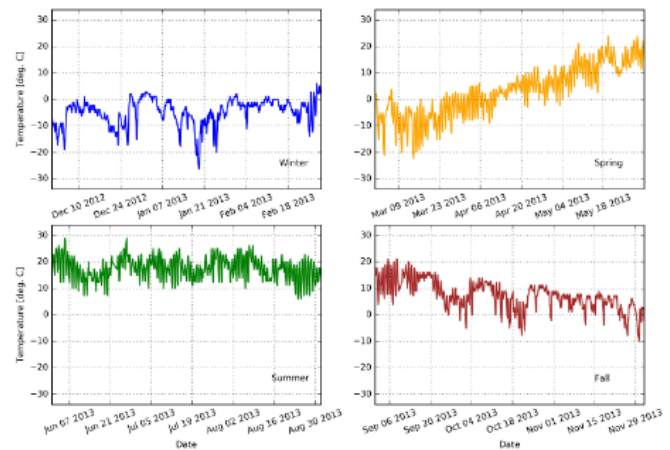
In this lesson, we'll explore the fundamentals of Matplotlib and learn how to create static plots for data visualization and analysis.



- Matplotlib is renowned for its versatility and ability to generate static, publication-quality plots in Python.
- In this lesson, participants will delve into the fundamentals of Matplotlib, gaining insight into its extensive capabilities for creating visually appealing plots.
- Through hands-on exercises, learners will master the art of crafting static plots using Matplotlib, honing their skills in data visualization and analysis.

Introduction to Matplotlib

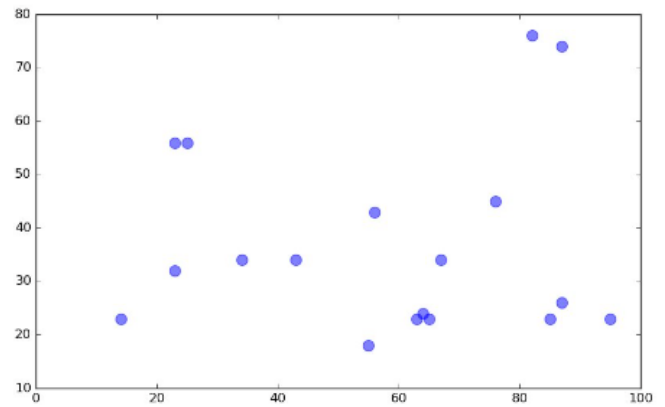
- Matplotlib is a comprehensive library for creating static, interactive, and animated plots in Python.
- It provides a wide range of plotting functions and customization options, making it suitable for various data visualization tasks.
- Matplotlib is widely used in scientific computing, data analysis, and exploratory data analysis.



- Matplotlib offers extensive support for creating static, interactive, and animated plots in Python, catering to diverse visualization needs across different domains.
- In addition to its rich set of plotting functions, Matplotlib provides a plethora of customization options, empowering users to fine-tune the appearance and style of their plots to suit specific requirements.
- Beyond its utility in scientific computing and data analysis, Matplotlib is a popular choice for exploratory data analysis (EDA), enabling users to visualize and explore datasets efficiently.
- Matplotlib's versatility extends to its integration with other Python libraries and frameworks, allowing seamless interoperability and expanding its capabilities for data visualization and analysis tasks.

Basic Plotting with Matplotlib

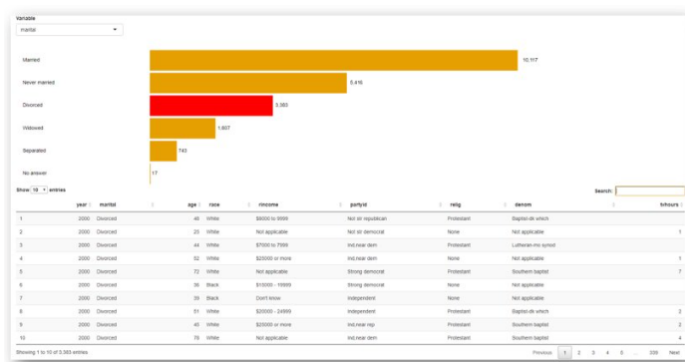
- Matplotlib provides a MATLAB-like interface for creating plots using a combination of functions and object-oriented methods.
- Some of the basic plot types supported by Matplotlib include line plots, scatter plots, bar charts, histograms, and pie charts.
- These plots can be customized with various parameters to control aspects such as colors, markers, labels, and axes.



- Matplotlib offers extensive documentation and a rich ecosystem of resources, including tutorials, examples, and community support, making it accessible to users of all levels.
- In addition to basic plot types, Matplotlib provides advanced visualization capabilities for creating complex plots such as 3D plots, contour plots, and heatmaps.
- Matplotlib integrates seamlessly with other Python libraries and frameworks, including NumPy, Pandas, and SciPy, enabling seamless data visualization and analysis workflows.
- The object-oriented approach in Matplotlib allows for fine-grained control over plot elements, making it suitable for creating publication-quality visualizations and customizing plots to meet specific requirements.

Customizing Plots

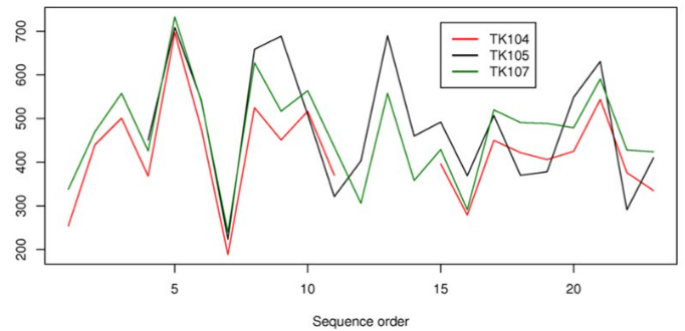
- Matplotlib allows users to customize plots extensively to meet specific requirements and preferences.
- Customization options include changing plot styles, adding annotations and text, adjusting axis limits and ticks, modifying colors and line styles, and incorporating legends and titles.
- Matplotlib provides fine-grained control over every aspect of plot appearance, enabling users to create visually appealing and informative plots.



- Matplotlib offers a plethora of customization options beyond basic styling, empowering users to tailor plots to their precise needs and preferences.
- Users can add annotations and text to highlight key features or provide additional context within plots, enhancing their communicative power.
- Fine-tuning axis limits and ticks allows for precise control over the scale and appearance of plots, ensuring clarity and readability.
- Matplotlib's extensive palette of colors and line styles, along with the ability to create custom color maps, provides flexibility in designing visually appealing plots.
- Incorporating legends and titles into plots helps users convey information effectively and guide interpretation.
- Modular architecture and object-oriented interface offer granular control over plot elements, facilitating the creation of complex and highly customized visualizations.

Saving Plots

- Once a plot is created, Matplotlib provides options for saving the plot in various formats, including PNG, PDF, SVG, and EPS.
- Users can specify the desired file format and resolution when saving plots, ensuring compatibility with different output formats and quality requirements.
- Saved plots can be used for presentations, reports, publications, and other purposes.



- Matplotlib offers a range of options for saving plots in different formats, such as PNG, PDF, SVG, and EPS, providing flexibility for various output requirements.
- Users can specify parameters like file format and resolution when saving plots, allowing for customization and optimization based on specific needs.
- Saved plots serve multiple purposes, including integration into presentations, inclusion in reports, publication in academic papers, and sharing with collaborators or stakeholders.

Conclusion

In conclusion, Matplotlib is a powerful library for creating static plots in Python.

By mastering the fundamentals of Matplotlib and exploring its customization options, analysts can create visually appealing and informative plots for data visualization and analysis.



Hands-On Exercise: For hands-on experience, review and complete *Exercise 1C* in the exercise guide for this course.

Additional Resources

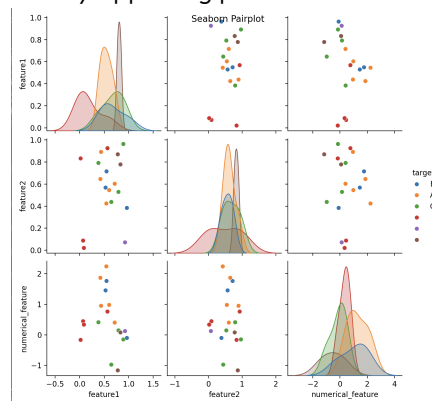
- Matplotlib Documentation: <https://matplotlib.org/>
- Matplotlib Gallery: <https://matplotlib.org/stable/gallery/index.html>

Lesson 1.4: Advanced Visualization Techniques with Seaborn



Seaborn is a Python data visualization library based on Matplotlib that provides a high-level interface for creating attractive and informative statistical graphics.

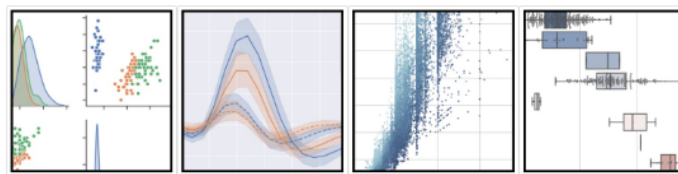
In this lesson, we'll explore advanced visualization techniques with Seaborn and learn how to create visually appealing plots for data analysis.



- Seaborn offers a higher-level interface compared to Matplotlib, simplifying the process of creating complex statistical graphics.
- It provides a wide range of built-in themes and color palettes, allowing users to create visually appealing plots effortlessly.
- Seaborn seamlessly integrates with Pandas DataFrames, making it convenient for visualizing data directly from DataFrame objects.
- Advanced visualization techniques covered in this lesson may include visualizing distributions, relationships between variables, and multivariate patterns using Seaborn's specialized functions.
- Through hands-on exercises, participants will gain proficiency in leveraging Seaborn's capabilities to produce insightful and aesthetically pleasing visualizations for data analysis.

Introduction to Seaborn

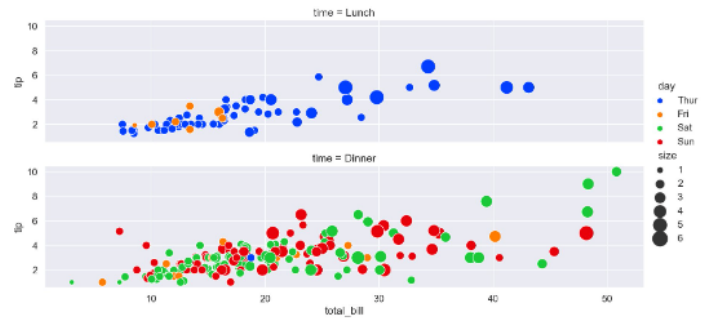
- Seaborn builds on top of Matplotlib and provides a simplified interface for creating complex statistical visualizations.
- It offers a wide range of built-in functions for creating various plot types, including scatter plots, bar plots, box plots, violin plots, heatmaps, and pair plots.
- Seaborn's default aesthetics are designed to produce visually appealing plots with minimal customization.



- Seaborn is built on top of Matplotlib, extending its capabilities by providing a simplified interface for generating complex statistical visualizations.
- In addition to its integration with Matplotlib, Seaborn offers a rich assortment of built-in functions tailored for creating diverse plot types, such as scatter plots, bar plots, box plots, violin plots, heatmaps, and pair plots.
- Seaborn's default aesthetics are carefully crafted to produce visually appealing plots with minimal customization, allowing users to create attractive visualizations effortlessly.
- Beyond its default settings, Seaborn provides extensive options for customization, enabling users to fine-tune plots according to specific requirements or preferences.

Styling and Aesthetics

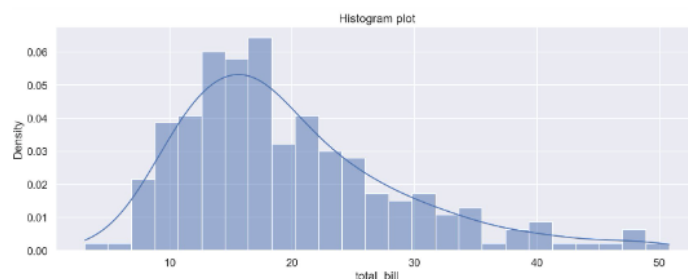
- One of Seaborn's key features is its support for easily customizing plot aesthetics to improve readability and visual appeal.
- Seaborn provides functions for changing plot styles, color palettes, font sizes, grid styles, and other visual elements.
- By adjusting these parameters, users can create plots that match their preferences and meet specific design requirements



- Seaborn's versatility extends to its ability to effortlessly customize plot aesthetics, enhancing readability and visual appeal.
- Beyond basic customization, Seaborn offers functions for fine-tuning plot styles, color palettes, font sizes, grid styles, and various other visual elements.
- With Seaborn's extensive parameter adjustment options, users can create visually stunning plots that align with their preferences and adhere to specific design standards.
- Seaborn's intuitive API allows for seamless exploration and experimentation with different aesthetic configurations, empowering users to find the perfect visual representation for their data.

Complex Plot Types

- Seaborn excels at creating complex statistical plots that visualize relationships between variables and highlight patterns in data.
- Some of the advanced plot types supported by Seaborn include:
 - Pair plots
 - Violin plots
 - Facet grids
 - Heatmaps



- **Pair Plots:** Pair plots visualize pairwise relationships between variables in a dataset, making them useful for exploring multivariate data.
- **Violin Plots:** Violin plots display the distribution of data across different categories, combining aspects of box plots and kernel density estimation.
- **Facet Grids:** Facet grids allow users to create multiple plots arranged in a grid layout, with each plot showing a different subset of the data based on categorical variables.
- **Heatmaps:** Heatmaps visualize data in a tabular format using colors to represent values, making them effective for displaying correlation matrices and other structured data.

Conclusion

In conclusion, Seaborn is a powerful library for creating advanced statistical visualizations in Python.

By leveraging its built-in functions and customization options, analysts can create visually appealing and informative plots to gain insights from data.



Hands-On Exercise: For hands-on experience, review and complete *Exercise 1D* in the exercise guide for this course.

Additional Resources

- Seaborn Documentation: <https://seaborn.pydata.org/>
- Seaborn Gallery: <https://seaborn.pydata.org/examples/index.html>

Module 2: Data Manipulation



Learning Objectives



Upon completion of this module, students should be able to:

- Understand the fundamentals of data manipulation using NumPy.
- Perform data manipulation and analysis tasks using Pandas.
- Explore the features and advantages of the Polars library for efficient data manipulation.

Lesson 2.1: Fundamentals of Data Manipulation using NumPy



NumPy is a fundamental package for scientific computing in Python, providing support for multidimensional arrays, mathematical functions, and linear algebra operations.

In this lesson, we'll explore the fundamentals of data manipulation using NumPy and learn how to perform common data manipulation tasks efficiently.

```
1
2 import numpy as np
3
4 # Create a NumPy array
5 arr = np.array([1, 2, 3, 4, 5, 6])
6
7 # Print the array
8 print("Original array:")
9 print(arr)
10
11 # Reshape the array to a 2x3 matrix
12 reshaped_arr = arr.reshape(2, 3)
13
14 # Print the reshaped array
15 print("\nReshaped array:")
16 print(reshaped_arr)
17
18 # Indexing: Accessing elements of the array
19 print("\nAccessing elements of the array:")
20 print("Element at index 0:", arr[0]) # Accessing the first element
21 print("Element at index 2:", arr[2]) # Accessing the third element
```

PROBLEMS 70 OUTPUT TERMINAL PORTS

```
(.venv) erikherman@Eriks-MacBook-Air Practicals % python module_2_lesson_1.py
Original array:
[1 2 3 4 5 6]

Reshaped array:
[[1 2 3]
 [4 5 6]]

Accessing elements of the array:
Element at index 0: 1
Element at index 2: 3

Accessing a subset of the array:
Elements from index 1 to 3: [2 3 4]
```

- NumPy serves as a cornerstone package for scientific computing in Python, offering robust support for multidimensional arrays, mathematical functions, and linear algebra operations.
- This lesson delves into the fundamental principles of data manipulation using NumPy, providing a solid foundation for performing various data manipulation tasks efficiently.
- Alongside basic array creation and manipulation, learners will also delve into advanced functionalities such as broadcasting, slicing, and reshaping arrays to suit specific analytical needs.
- NumPy's vectorized operations and optimized routines contribute to enhanced computational efficiency, making it a preferred choice for handling large datasets and complex mathematical computations.

Introduction to NumPy

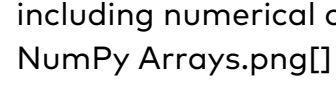
- NumPy, short for Numerical Python, is a powerful library for numerical computing in Python.
- It provides support for creating and manipulating multidimensional arrays (ndarrays), which are the foundation for many data manipulation and analysis tasks.
- NumPy arrays are more efficient than Python lists for storing and manipulating large datasets, making them well-suited for numerical computations and data processing tasks.

NUMPY ARRAYS STORE NUMERIC DATA IN A ROW-AND-COLUMN STRUCTURE

4	12	0	4
6	11	8	4
18	14	13	7

- NumPy is widely used in scientific computing, machine learning, data analysis, and other fields due to its efficiency and versatility.
- In addition to arrays, NumPy offers a wide range of mathematical functions and operations for performing advanced numerical computations.
- NumPy arrays support broadcasting, a powerful feature that simplifies operations on arrays with different shapes by automatically aligning them.

Creating NumPy Arrays

- NumPy arrays can be created from Python lists or tuples using the `np.array()` function.
 - NumPy also provides functions for generating arrays with specific shapes and values, such as `np.zeros()`, `np.ones()`, `np.arange()`, and `np.linspace()`.
 - These functions allow users to create arrays quickly and easily for various purposes, including numerical computations, data analysis, and machine learning.
-
- NumPy arrays offer efficient storage and manipulation of homogeneous data, making them ideal for numerical computations and data analysis tasks.
 - In addition to basic creation methods, NumPy provides functions like `np.random.rand()` and `np.random.randn()` for generating arrays with random values.
 - NumPy arrays support multidimensional data structures, enabling representation of matrices, tensors, and higher-dimensional arrays.

Indexing and Slicing

- NumPy arrays support indexing and slicing operations for accessing and modifying elements within arrays.
- Array elements can be accessed using square brackets `[]`, and array slices can be obtained using colon `:` notation.
- NumPy also supports advanced indexing techniques, such as boolean indexing and fancy indexing, for selecting subsets of array elements based on specific conditions or criteria.

```
>>> a[0, 3:5]
array([3, 4])

>>> a[4:, 4:]
array([[44, 55],
       [54, 55]])

>>> a[:, 2]
a([2, 12, 22, 32, 42, 52])

>>> a[2::2, ::2]
array([[20, 22, 24],
       [40, 42, 44]])
```

0	1	2	3	4	5
10	11	12	13	14	15
20	21	22	23	24	25
30	31	32	33	34	35
40	41	42	43	44	45
50	51	52	53	54	55

- Indexing and slicing operations in NumPy arrays enable efficient access to and modification of array elements.
- NumPy arrays support multidimensional indexing and slicing, allowing for manipulation of subarrays along different axes.
- Array elements can be modified directly using indexing and slicing operations, facilitating in-place updates and transformations.

Array Operations

- NumPy arrays support various mathematical and statistical operations, including element-wise arithmetic operations, array broadcasting, and linear algebra operations.
- NumPy provides a wide range of built-in functions for performing mathematical operations on arrays, such as addition, subtraction, multiplication, division, exponentiation, trigonometric and statistical functions.

```
Matrix A:  
[[0 3 7 4 6 9]  
 [2 6 7 4 3 7]  
 [7 2 5 4 1 7]  
 [5 1 4 0 9 5]  
 [8 0 9 2 6 3]], shape=(5, 6)  
  
Matrix B:  
[[8 2 4]  
 [2 6 4]  
 [8 6 1]], shape=(3, 3)  
  
Matrix A after submatrix multiplication:  
[[ 0  3  7  4  6  9]  
 [ 2  6 88 56 47  7]  
 [ 7  2 56 40 37  7]  
 [ 5  1104 62 25  5]  
 [ 8  0  9  2  6  3]], shape=(5, 6)
```

- NumPy arrays facilitate efficient mathematical and statistical operations, empowering users with capabilities like element-wise arithmetic operations, array broadcasting, and linear algebra computations.
- Beyond basic arithmetic, NumPy boasts an extensive library of built-in functions tailored for array manipulation, encompassing arithmetic operations, exponentiation, trigonometric calculations, and a plethora of statistical functions.
- The array broadcasting feature in NumPy enables the automatic alignment and computation of operations between arrays of different shapes, enhancing flexibility and ease of use in array operations.
- NumPy's support for linear algebra operations enables users to perform matrix multiplication, inversion, eigenvalue computations, and other advanced mathematical operations efficiently.

Conclusion

In conclusion, NumPy is a fundamental library for data manipulation and numerical computing in Python.

By mastering the basics of NumPy arrays and operations, analysts can efficiently perform common data manipulation tasks and conduct numerical computations for various applications.



Hands-On Exercise: For hands-on experience, review and complete *Exercise 2A* in the exercise guide for this course.

Additional Resources

- NumPy Documentation: <https://numpy.org/doc/>
- NumPy Quickstart Tutorial: <https://numpy.org/doc/stable/user/quickstart.html>

Lesson 2.2: Data Manipulation and Analysis using Pandas



Pandas is a powerful Python library for data manipulation and analysis, built on top of NumPy.

In this lesson, we'll explore the capabilities of Pandas and learn how to use it to perform various data manipulation and analysis tasks efficiently.

```
1
2 # Step 1: Import necessary libraries
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 # Step 2: Generate some sample data
7 # For this example, let's generate 1000 random numbers from a normal distribution
8 data = np.random.normal(loc=0, scale=1, size=1000)
9
10 # Step 3: Create the Matplotlib histogram plot
11 plt.figure(figsize=(8, 6))
12 plt.hist(data, bins=30, color='skyblue', edgecolor='black')
13
14 # Step 4: Customize the plot (optional)
15 plt.title('Distribution of a Numerical Variable')
16 plt.xlabel('Value')
17 plt.ylabel('Frequency')
18
19 # Step 5: Show the plot
20 plt.show()
```

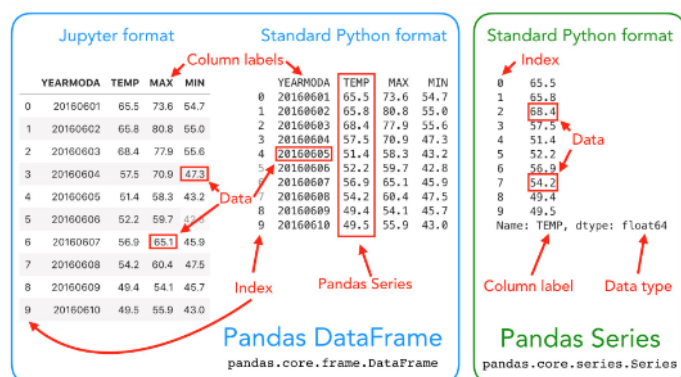
```
First few rows of the dataset:
Region Product Category Sales Amount
0 North A 100
1 North B 200
2 South A 150
3 South C 300
4 West B 250

Total sales amount by product category in the North region:
Product Category
A 100
B 200
Name: Sales Amount, dtype: int64
```

- Pandas offers high-level data structures and functions designed for efficient data manipulation and analysis, providing a user-friendly interface for working with structured data.
- Leveraging its DataFrame and Series data structures, Pandas facilitates tasks such as data cleaning, transformation, aggregation, and exploration.
- In addition to basic data manipulation operations, Pandas supports advanced functionality including handling missing data, merging and joining datasets, and time series analysis.
- The lesson covers various Pandas functionalities and methods, including indexing and selection, groupby operations, reshaping data, and working with datetime objects.

Introduction to Pandas

- Pandas provides data structures and functions for working with structured data, making it ideal for tasks such as data cleaning, transformation, exploration, and analysis.
- The two main data structures in Pandas are Series and DataFrame, which represent one-dimensional and two-dimensional labeled arrays, respectively.
- Pandas simplifies common data manipulation tasks by providing intuitive and expressive syntax for performing operations on data.



- Pandas offers a wide range of functions and methods for working with structured data, facilitating tasks such as data cleaning, transformation, exploration, and analysis with ease and efficiency.
- In addition to Series and DataFrame, Pandas provides other data structures such as Index and Panel, expanding its capabilities for handling various types of data.
- Its robust indexing capabilities enable users to access, manipulate, and analyze data efficiently, whether through label-based, integer-based, or boolean-based indexing.
- Pandas integrates seamlessly with other Python libraries and tools commonly used in data analysis workflows, such as NumPy, Matplotlib, and scikit-learn, enhancing its versatility and interoperability.
- Its extensive documentation and vibrant community support provide resources and guidance for users at all levels, from beginners to advanced practitioners, fostering learning and collaboration within the data science community.

Key Features of Pandas

- Data Structures
- Data Alignment
- Indexing and Selection
- Data Cleaning and Transformation
- Data Analysis and Visualization



- **Efficient Computation:** Pandas leverages vectorized operations and optimized algorithms to perform computations efficiently, even on large datasets.
- **Time Series Functionality:** Pandas offers robust support for working with time series data, including date/time indexing, resampling, and time zone handling.
- **Flexible I/O Tools:** Pandas provides versatile tools for reading and writing data from various file formats, including CSV, Excel, SQL databases, JSON, and more.
- **Integration with Database Systems:** Pandas seamlessly integrates with database systems like SQL databases, enabling users to query, manipulate, and analyze data directly from databases.
- **Powerful GroupBy Functionality:** Pandas' GroupBy functionality allows for flexible data aggregation and analysis by grouping data based on one or more keys and applying functions to each group.

Working with Pandas

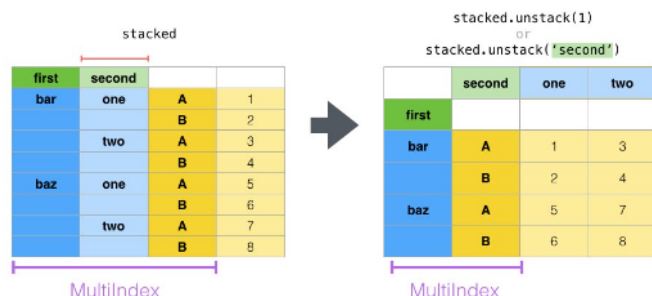
In this section, we'll cover some common data manipulation and analysis tasks using Pandas:

Loading and Reading Data

Data Cleaning and Transformation

Data Analysis and Visualization

- Calculate descriptive statistics for data using functions like `mean()`, `median()`, `std()`, and `describe()`.
- Create visualizations and plots directly from Pandas data structures using libraries like Matplotlib and Seaborn



- Pandas provides extensive support for loading data from various file formats, including CSV, Excel, JSON, and SQL databases, making it versatile for diverse data sources.
- When loading data with Pandas, users can customize the process by specifying column names, data types, and parsing dates, ensuring flexibility and accuracy in data ingestion.
- In addition to handling missing data with methods like `dropna()` or `fillna()`, Pandas offers advanced techniques such as interpolation and imputation to address missing values effectively.
- Pandas' `drop_duplicates()` function allows users to remove duplicate rows from datasets, ensuring data integrity and consistency.
- Beyond basic data manipulation tasks, Pandas empowers users to perform complex data transformations such as sorting, filtering, and grouping with ease, facilitating comprehensive data analysis workflows.

Conclusion

In conclusion, Pandas is a powerful tool for data manipulation and analysis in Python.

By mastering the capabilities of Pandas and practicing with real-world datasets, analysts can efficiently perform data manipulation tasks, conduct exploratory data analysis, and derive valuable insights from data.



Hands-On Exercise: For hands-on experience, review and complete *Exercise 2B* in the exercise guide for this course.

Additional Resources

- Pandas Documentation: <https://pandas.pydata.org/docs/>
- Pandas User Guide: https://pandas.pydata.org/docs/user_guide/index.html

Lesson 2.3: Features of Polars for Efficient Data Manipulation



Polars is a fast and efficient data manipulation library for Python and Rust, designed for processing large-scale tabular data.

In this lesson, we'll explore the key features of Polars and learn how to leverage them for efficient data manipulation tasks.

```
Loading time (Pandas): 0.0034761428833007812
Loading time (NumPy): 0.00030493736267089844
Loading time (Polars): 0.0019021034240722656

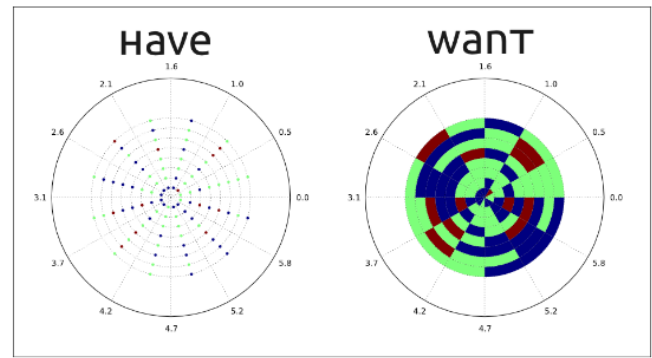
Filtering time (Pandas): 0.0034759044647216797
Filtering time (NumPy): 1.3113021850585938e-05
Filtering time (Polars): 0.0015671253204345703

Aggregation time (Pandas): 0.0029969215393066406
Aggregation time (NumPy): 1.5735626220703125e-05
Aggregation time (Polars): 0.00092315673828125
```

- Polars is not only a Python library but also has Rust bindings, offering performance benefits by leveraging Rust's speed and memory safety.
- It excels in handling large-scale tabular data efficiently, making it suitable for processing massive datasets with ease.
- Polars provides a wide range of operations for data manipulation tasks, including filtering, aggregation, joining, and transformation, enabling users to perform complex data transformations effortlessly.
- With its DataFrame abstraction, Polars simplifies data manipulation tasks by providing a familiar and intuitive interface similar to pandas DataFrames.
- Polars supports parallel and vectorized processing, utilizing multi-threading and SIMD (Single Instruction, Multiple Data) instructions to accelerate computations and enhance performance.

Introduction to Polars

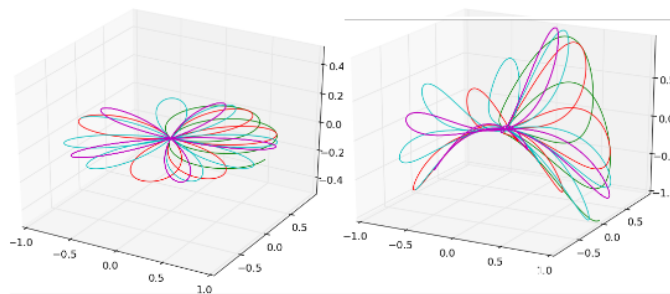
- Polars is built with a focus on performance, scalability, and ease of use, making it suitable for handling large datasets with millions or even billions of rows.
- It provides a DataFrame abstraction similar to Pandas but with optimizations for speed and memory efficiency.
- Polars is implemented in Rust, with a Python API for seamless integration with the Python ecosystem.



- Polars emphasizes performance, scalability, and ease of use, making it adept at processing large datasets comprising millions or even billions of rows.
- It offers a DataFrame abstraction akin to Pandas, yet distinguishes itself with optimizations geared towards accelerating computations and enhancing memory efficiency.
- Implemented in Rust, Polars provides a Python API for effortless integration within the Python ecosystem, enabling seamless interoperability with other Python libraries and tools.
- Polars boasts advanced functionalities, including parallel processing capabilities and support for lazy evaluation, further enhancing its performance and scalability for data processing tasks.

Key Features of Polars

- **Lazy Evaluation**
- **Columnar Storage**
- **Parallel Processing**
- **Out-of-Core Processing**
- **API Compatibility**
- **Integration with Arrow**



- **Lazy Evaluation:** Polars employs lazy evaluation to optimize query execution and minimize memory usage. Operations are deferred until necessary, enabling efficient processing of large datasets without loading the entire dataset into memory.
- **Columnar Storage:** Polars stores data in a columnar format, enhancing cache locality and facilitating vectorized processing.
- **Parallel Processing:** Leveraging multi-threading and SIMD instructions, Polars executes data processing operations in parallel. This parallelism harnesses modern multi-core processors, resulting in high throughput for data manipulation tasks.
- **Out-of-Core Processing:** Polars supports out-of-core processing, accommodating datasets that exceed available memory by spilling data to disk.
- **API Compatibility:** Polars provides a Pandas-like API, easing the transition for users familiar with Pandas.
- **Integration with Arrow:** Polars integrates with Apache Arrow, facilitating interoperability with other Arrow-compatible libraries and ecosystems such as PyArrow and Arrow Flight.

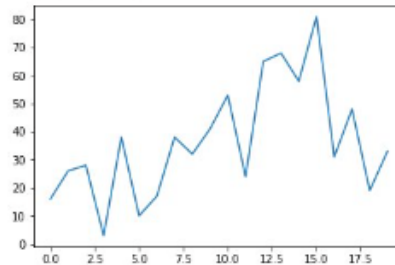
Working with Polars

Loading and Reading Data

Data Manipulation and Transformation:

Parallel Processing and Optimization:

```
In [1]: %matplotlib inline
In [2]: import matplotlib.pyplot as plt
In [3]: import random
In [4]: data = [random.randint(0, 100) for i in range(20)]
In [5]: plt.plot(data)
Out[5]: [<matplotlib.lines.Line2D at 0x7fbf8f04b160>]
```



- **Loading and Reading Data:**

- Polars offers versatile capabilities for loading data from various file formats such as CSV, Parquet, Arrow, and Apache Avro.
- Users can customize data loading options in Polars, including specifying column types, delimiters, and compression methods, to suit their specific requirements.

- **Data Manipulation and Transformation:**

- With Polars' DataFrame API, users can seamlessly perform common data manipulation tasks such as filtering, selecting, aggregating, and joining datasets.
- Polars leverages lazy evaluation and out-of-core processing techniques, enabling efficient handling of large datasets that surpass available memory constraints.

- **Parallel Processing and Optimization:**

- Polars leverages multi-threading and SIMD (Single Instruction, Multiple Data) instructions for parallel execution of data processing operations, enhancing throughput and performance.
- Users can optimize data processing pipelines in Polars by employing techniques like predicate pushdown and columnar processing to minimize computational overhead and improve efficiency.

Conclusion

In conclusion, Polars is a powerful library for efficient data manipulation and processing in Python and Rust.

By leveraging its key features and optimizations, analysts can efficiently handle large-scale datasets and perform complex data manipulation tasks with ease.



Hands-On Exercise: For hands-on experience, review and complete *Exercise 2C* in the exercise guide for this course.

Additional Resources

- Polars Documentation: <https://pola.rs/>
- Polars GitHub Repository: <https://github.com/pola-rs/polars>

Module 3: Statistical Analysis



Learning Objectives



Upon completion of this module, students should be able to:

- Understand the principles of statistical analysis.
- Learn to use Statsmodels for regression analysis, hypothesis testing, and other statistical tasks.
- Explore advanced statistical analysis techniques using Pingouin.
- Learn to use SciPy for statistical computations and hypothesis testing.

Lesson 3.1: Principles of Statistical Analysis



Statistical analysis is a fundamental component of data science and plays a crucial role in deriving meaningful insights from data.

In this lesson, we'll explore the principles of statistical analysis and learn about key concepts and techniques used to analyze data and draw conclusions from it.

```
First few rows of the dataset:
 1 10 20 A
0 2 15 25 B
1 3 20 30 C
2 4 25 35 A
3 5 30 40 B
4 6 35 45 C

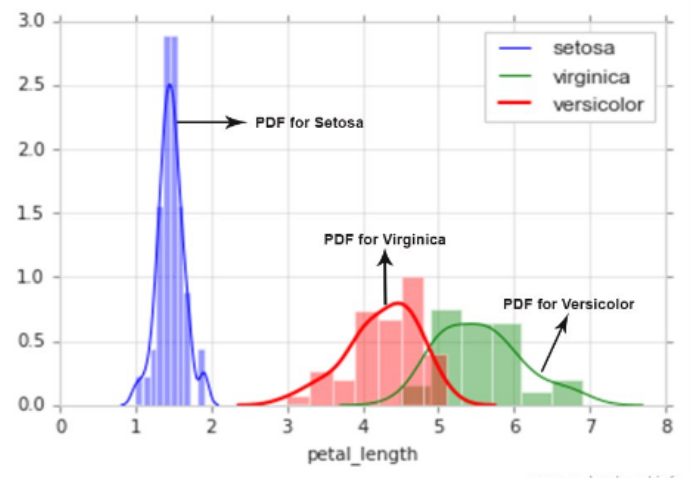
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0    1         9 non-null    int64
1    10        9 non-null    int64
2    20        9 non-null    int64
3    A         9 non-null    object
dtypes: int64(3), object(1)
memory usage: 420.0+ bytes
None

Descriptive statistics:
              1      10      20
count  9.000000  9.000000  9.000000
mean    6.000000  35.000000  45.000000
std     2.738613  13.693064  13.693064
```

- Statistical analysis forms the backbone of data science, offering essential tools and methodologies for extracting valuable insights from data.
- This lesson delves into the foundational principles of statistical analysis, providing learners with a comprehensive understanding of the underlying concepts and techniques.
- Learners will explore key statistical concepts, including probability distributions, hypothesis testing, and regression analysis, to gain proficiency in analyzing and interpreting data.
- Through practical examples and exercises, participants will apply statistical techniques to real-world datasets, honing their analytical skills and problem-solving abilities.

Introduction to Statistical Analysis

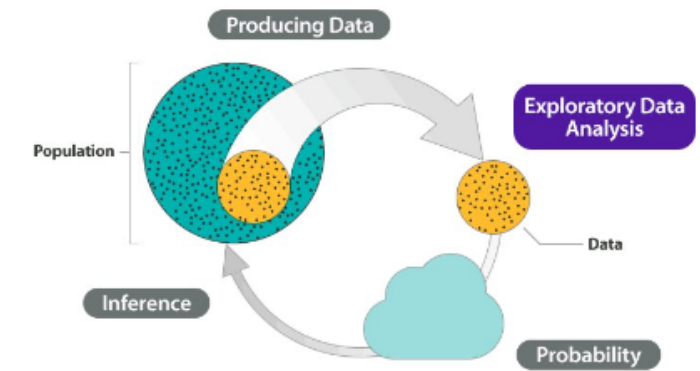
- Statistical analysis involves the collection, interpretation, and presentation of data to uncover patterns, relationships, and trends.
- It encompasses a wide range of techniques for summarizing data, testing hypotheses, estimating parameters, and making predictions.
- Statistical analysis helps researchers and analysts make informed decisions and draw reliable conclusions based on empirical evidence.



- Statistical analysis plays a crucial role in various fields, including science, business, healthcare, and social sciences, aiding in evidence-based decision-making and problem-solving.
- It involves both descriptive and inferential statistics, where descriptive statistics summarize data characteristics while inferential statistics draw conclusions about populations based on sample data.
- Statistical analysis techniques encompass a diverse array of methods, such as regression analysis, hypothesis testing, ANOVA (analysis of variance), and time series analysis, among others.

Key Concepts in Statistical Analysis

- Descriptive Statistics
- Inferential Statistics
- Probability Theory
- Statistical Models



- **Sampling Methods:** Sampling methods are techniques used to select a subset of data from a larger population for analysis. Common methods include random sampling, stratified sampling, and cluster sampling.
- **Parametric vs. Non-parametric Methods:** Statistical methods can be classified as parametric or non-parametric based on assumptions about the underlying data distribution. Parametric methods assume a specific distribution (e.g., normal distribution), while non-parametric methods do not make such assumptions.
- **Experimental Design:** Experimental design involves planning and conducting experiments to collect data in a systematic and controlled manner. It includes considerations such as sample size determination, randomization, and control of confounding variables.
- **Statistical Significance:** Statistical significance refers to the likelihood that an observed effect in a dataset is not due to chance. It is often assessed using hypothesis testing and p-values, with lower p-values indicating greater statistical significance.

Principles of Statistical Analysis

In this section, we'll cover some fundamental principles of statistical analysis:

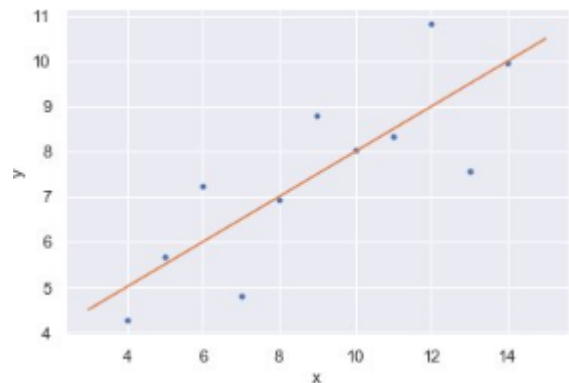
Principle 1: Define the Research Question

Principle 2: Choose the Right Data

Principle 3: Understand the Data Distribution

Principle 4: Apply Appropriate Statistical Techniques

Principle 5: Interpret Results Carefully



- **Principle 1: Define the Research Question**

- Before embarking on any statistical analysis, it is imperative to define the research question or objective clearly.
- A well-defined research question guides the selection of appropriate statistical techniques and provides a framework for interpreting results accurately.

- **Principle 2: Choose the Right Data**

- The selection of suitable data is paramount for conducting meaningful statistical analysis.
- Ensure that the data chosen is pertinent to the research question, representative of the population of interest, and devoid of biases or errors.

- **Principle 3: Understand the Data Distribution**

- A profound understanding of the distribution of data is crucial for selecting suitable statistical methods and ensuring accurate interpretation of results.

- **Principle 4: Apply Appropriate Statistical Techniques**

- It is essential to choose statistical techniques that align with the type of data and research question at hand.

- **Principle 5: Interpret Results Carefully**

- The interpretation of statistical findings necessitates meticulous attention to context, assumptions, and constraints.

Conclusion

In conclusion, statistical analysis is a powerful tool for extracting insights from data and making informed decisions.

By following fundamental principles and best practices in statistical analysis, researchers and analysts can conduct rigorous and reliable data analysis and contribute to evidence-based decision-making.



Hands-On Exercise: For hands-on experience, review and complete *Exercise 3A* in the exercise guide for this course.

Additional Resources

- "Introduction to Statistical Learning" by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani
- "The Art of Statistics: Learning from Data" by David Spiegelhalter
- Coursera: Statistics with R Specialization: <https://www.coursera.org/specializations/statistics>

Lesson 3.2: Regression Analysis and Hypothesis Testing with Statsmodels



Regression analysis and hypothesis testing are essential techniques in statistical analysis for exploring relationships between variables and making inferences from data.

In this lesson, we'll explore how to perform regression analysis and hypothesis testing using the Statsmodels library in Python.

```

OLS Regression Results
=====
Dep. Variable: target    R-squared:    nan
Model: OLS             Adj. R-squared: nan
Method: Least Squares   F-statistic: nan
Date: Wed, 21 Feb 2024   Prob(F-statistic): nan
Time: 09:38:00          Log-Likelihood: nan
No. Observations: 5     AIC: nan
Df Residuals: 2         BIC: nan
Df Model: 2
Covariance Type: nonrobust

             coef    std err          t      Pr>|t|    [0.025    0.975]
-----
const      nan         nan         nan         nan         nan         nan
feature1    nan         nan         nan         nan         nan         nan
feature2    nan         nan         nan         nan         nan         nan

Omnibus: nan      Durbin-Watson: nan
Prob(Omnibus): nan   Jarque-Bera (JB): nan
Skew: nan      Prob(JB): nan
Kurtosis: nan      Cond. No.      68.1

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Hypothesis Testing:
Test for feature1 coefficient:
Null Hypothesis: Coefficient for feature1 is zero
Alternative Hypothesis: Coefficient for feature1 is not zero
p-values: nan
Conclusions: Reject the null hypothesis if p-value < significance level (e.g., 0.05)

```

- Regression analysis is a statistical technique used to model and analyze the relationships between one or more independent variables (predictors) and a dependent variable (outcome).
- Hypothesis testing is a statistical method used to make inferences about population parameters based on sample data, allowing researchers to assess the significance of observed effects.
- Statsmodels is a Python library that provides classes and functions for performing various statistical analyses, including regression analysis and hypothesis testing.
- Participants will also explore hypothesis testing techniques using Statsmodels, including t-tests, ANOVA, and chi-square tests, to evaluate hypotheses and make informed decisions based on statistical evidence.
- Practical applications of regression analysis and hypothesis testing include predicting outcomes, identifying relationships between variables, testing research hypotheses, and making data-driven decisions in various domains such as finance, healthcare, and social sciences.

Introduction to Statsmodels

- Statsmodels is a Python library for statistical modeling and hypothesis testing.
- It provides a wide range of tools for estimating and interpreting various statistical models, including linear regression, logistic regression, time series analysis, and more.
- Statsmodels integrates seamlessly with other scientific computing libraries such as NumPy and Pandas, making it a powerful tool for data analysis in Python.

```

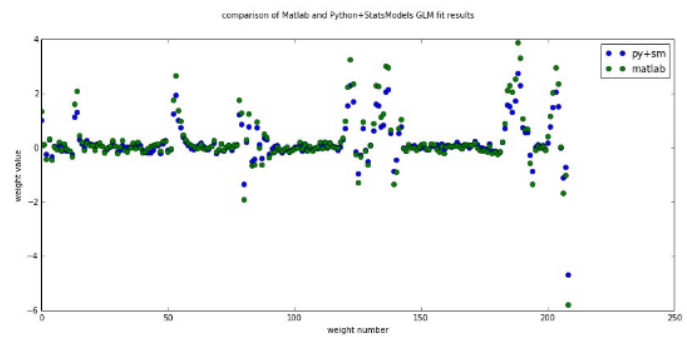
ot respect the domain of the Gamma family.
DomainWarning)
Generalized Linear Model Regression Results
=====
Dep. Variable:          y      No. Observations:      32
Model:                GLM      DF Residuals:         24
Model Family:          Gamma    DF Model:           7
Link Function:         inverse_power    Scale:           0.00358428317349
Method:                IRLS     Log-Likelihood:      -83.017
Date:                  Sun, 18 Feb 2018    Deviance:         0.087389
Time:                  15:24:11    Pearson chi2:     0.0860
No. Iterations:        4
=====
               coef      std err          z      P>|z|      [0.025      0.975]
-----
const         -0.0178      0.011      -1.548      0.122      -0.040      0.005
x1             4.902e-05      1.62e-05      3.000      0.002      1.78e-05      8.14e-05
x2              0.0020      0.001      3.024      0.000      0.001      0.003
x3            -7.181e-05      2.71e-05      -2.648      0.008      -0.000     -1.87e-05
x4              0.0001      4.86e-05      2.757      0.006      3.23e-05      0.000
x5            -1.468e-07      1.24e-07      -1.187      0.235     -3.89e-07      9.56e-08
x6             -0.0005      0.000      -2.159      0.031     -0.001     -4.78e-05
x7            -2.427e-06      7.46e-07      -3.253      0.001     -3.89e-06     -9.65e-07
=====

```

- Statsmodels offers comprehensive support for conducting statistical modeling and hypothesis testing, encompassing various techniques beyond linear and logistic regression.
- Its extensive toolkit includes capabilities for time series analysis, generalized linear models, survival analysis, and nonparametric methods, among others, providing a broad spectrum of statistical analysis tools.
- Statsmodels boasts a user-friendly interface and intuitive API, facilitating straightforward model estimation, interpretation, and diagnostics for both novice and experienced users.
- Beyond its core functionality, Statsmodels continues to evolve, with ongoing development efforts aimed at enhancing performance, expanding capabilities, and improving user experience.

Key Features of Statsmodels

- **Linear Regression**
- **Hypothesis Testing**
- **Diagnostic Tests**
- **Model Visualization**
- **Integration with Pandas**



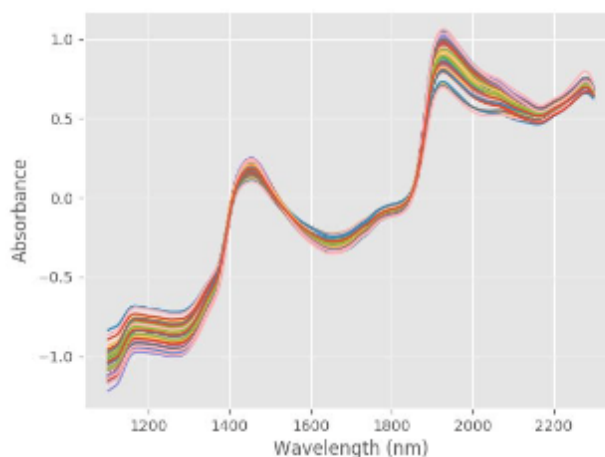
- **Time Series Analysis:** Statsmodels offers extensive support for time series analysis, including classes and functions for modeling time series data, forecasting, and performing diagnostic tests.
- **Econometric Models:** Statsmodels includes a wide range of econometric models and methods for analyzing economic and financial data, such as autoregressive integrated moving average (ARIMA) models, vector autoregression (VAR) models, and panel data analysis.
- **Survival Analysis:** Statsmodels supports survival analysis, offering classes and functions for modeling time-to-event data, estimating survival curves, and conducting hypothesis tests in survival analysis.
- **Generalized Linear Models (GLM):** In addition to linear regression, Statsmodels supports generalized linear models (GLM) for analyzing non-normal and non-linear data, including logistic regression, Poisson regression, and gamma regression.
- **Model Comparison and Selection:** Statsmodels provides tools for comparing and selecting between different models.

Regression Analysis with Statsmodels

In this section, we'll cover how to perform regression analysis using Statsmodels:

Simple Linear Regression - Use Statsmodels to fit a simple linear regression model to data and interpret the results. - Assess the statistical significance of the regression coefficients using hypothesis testing. - Visualize the regression line and predicted values using matplotlib.

Multiple Linear Regression - Extend simple linear regression to multiple linear regression models with multiple predictor variables. - Interpret the coefficients of multiple regression models and assess their statistical significance. - Use diagnostic tests and visualizations to evaluate the assumptions of multiple regression models.



- Statsmodels offers robust capabilities for performing regression analysis, facilitating the exploration of relationships between variables in datasets.
- Simple linear regression, a fundamental technique in regression analysis, involves fitting a linear model to data with a single predictor variable.
- Through hypothesis testing, Statsmodels enables users to assess the statistical significance of regression coefficients, aiding in the interpretation of model results.
- Visualizations created using matplotlib complement regression analysis by illustrating the regression line and predicted values, enhancing understanding and interpretation.
- Multiple linear regression expands upon simple linear regression by incorporating multiple predictor variables into the model, allowing for more comprehensive analysis of relationships between variables.
- Interpretation of coefficients in multiple regression models involves understanding the impact of each predictor variable on the outcome variable while considering the effects of other variables in the model.
- Diagnostic tests and visualizations serve as valuable tools for evaluating the assumptions underlying multiple regression models, helping to ensure the validity of model results and conclusions.

Hypothesis Testing with Statsmodels

In this section, we'll cover how to perform hypothesis testing using Statsmodels:

Testing Model Coefficients

Goodness-of-Fit Tests

		Reality	
		H_0 False	H_0 True
Test	Reject H_0	✓ Correct rejection H_1 - Power = $1 - \beta$	✗ Type I error = α
	Accept H_0	✗ Type II error	✓ Correct acceptance of H_0

- Statsmodels facilitates hypothesis testing by allowing users to assess the statistical significance of coefficients in regression models.
- T-tests and F-tests are commonly used for hypothesis testing, enabling evaluation of individual coefficients and overall model fit, respectively.
- Interpretation of hypothesis test results involves analyzing p-values and confidence intervals to make informed decisions about the significance of model coefficients.
- Goodness-of-fit tests, including metrics like the R-squared statistic and the F-statistic, assess the overall fit of regression models.
- By interpreting the results of goodness-of-fit tests, analysts can gauge the explanatory power of regression models and determine their effectiveness in capturing the variability of the data.

Conclusion

In conclusion, Statsmodels is a powerful library for regression analysis and hypothesis testing in Python.

By leveraging its features and capabilities, analysts can explore relationships between variables, test hypotheses, and make informed decisions based on empirical evidence.



Hands-On Exercise: For hands-on experience, review and complete *Exercise 3B* in the exercise guide for this course.

Additional Resources

- Statsmodels Documentation: <https://www.statsmodels.org/stable/index.html>
- "Regression Analysis: A Constructive Critique" by Richard B. Darlington
- Coursera: Regression Models: <https://www.coursera.org/learn/regression-models>

Lesson 3.3: Advanced Statistical Analysis Techniques with Pingouin



Pingouin is a Python library for statistical analysis built on top of Pandas and SciPy. It provides a wide range of advanced statistical tests and functions for exploring relationships between variables and conducting hypothesis testing.

In this lesson, we'll explore how to use Pingouin for advanced statistical analysis techniques.

```
Pearson correlation coefficient:
      n      r      CI95%      p-val      BF10      power
pearson 5  1.0  [1.0, 1.0]      0.0      inf      1

Spearman correlation coefficient:
      n      r      CI95%      p-val      power
spearman 5  1.0  [1.0, 1.0]  1.404265e-24      1.0

Kendall correlation coefficient:
      n      r      CI95%      p-val      power
kendall 5  1.0  [1.0, 1.0]  0.016667      1.0

Wilcoxon signed-rank test:
      W-val alternative      p-val      RBC      CLES
Wilcoxon  0.0      two-sided  0.0625  -1.0      0.2

Mann-Whitney U test:
      U-val alternative      p-val      RBC      CLES
MWU      5.0      two-sided  0.141238  0.6      0.2
```

- Pingouin is not only built on top of Pandas and SciPy but also integrates seamlessly with other Python libraries commonly used in data analysis and machine learning workflows.
- In addition to basic statistical tests, Pingouin offers advanced statistical methods such as repeated measures ANOVA, mixed-design ANOVA, and Bayesian correlation analysis.
- Pingouin provides a user-friendly interface for conducting statistical analyses, making it accessible to users with varying levels of statistical expertise.
- The library includes comprehensive documentation and examples, facilitating the learning and adoption of advanced statistical techniques.

Introduction to Pingouin

- Pingouin is designed to simplify the process of conducting advanced statistical analysis in Python.
 - It offers a user-friendly interface and integrates seamlessly with Pandas DataFrames, making it easy to perform complex statistical tests and interpret the results.
 - Pingouin provides support for various statistical tests, including parametric and non-parametric tests, correlation analysis, and effect size estimation.
-
- Pingouin streamlines advanced statistical analysis in Python by providing an intuitive and accessible framework.
 - Its seamless integration with Pandas DataFrames facilitates the execution of complex statistical tests directly on structured data, enhancing workflow efficiency.
 - Pingouin offers a comprehensive range of statistical tests, encompassing parametric and non-parametric tests, correlation analysis, and effect size estimation, catering to diverse analytical needs.
 - In addition to its statistical testing capabilities, Pingouin provides functionalities for descriptive statistics, distribution fitting, and power analysis, empowering users with a comprehensive statistical toolkit.
 - Pingouin prioritizes transparency and reproducibility by implementing best practices in statistical analysis and providing detailed documentation and tutorials to guide users through the analysis process.

Key Features of Pingouin

- **Comprehensive Test Suite**
 - **Effect Size Estimation**
 - **Power Analysis**
 - **Robust Statistical Tests**
 - **Visualization Tools**
-

- **Assumption Testing:** Pingouin provides functions for testing assumptions required for various statistical tests, such as normality and homogeneity of variances.
- **Non-Parametric Tests:** In addition to parametric tests, Pingouin offers a wide range of non-parametric tests for analyzing data that does not meet the assumptions of parametric tests.
- **Multi-Comparison Correction:** Pingouin supports multiple comparison correction methods to account for the increased risk of Type I errors when conducting multiple comparisons.
- **Integration with Pandas:** Pingouin seamlessly integrates with Pandas DataFrames, allowing users to perform statistical analysis directly on Pandas data structures.
- **Documentation and Support:** Pingouin provides comprehensive documentation and user support, including tutorials, examples, and a dedicated community forum for users to seek assistance and share insights.

Advanced Statistical Analysis Techniques

In this section, we'll cover some advanced statistical analysis techniques using Pingouin:

Parametric and Non-Parametric Tests

Correlation Analysis

Effect Size Estimation

```
Pearson correlation coefficient:
n      r      CI95%    p-val BF10    power
pearson 5 1.0 [1.0, 1.0] 0.0 inf      1

Spearman correlation coefficient:
n      r      CI95%    p-val    power
spearman 5 1.0 [1.0, 1.0] 1.404265e-24 1.0

Kendall correlation coefficient:
n      r      CI95%    p-val    power
kendall 5 1.0 [1.0, 1.0] 0.016667 1.0

Wilcoxon signed-rank test:
W-val alternative    p-val RBC CLES
Wilcoxon 0.0 two-sided 0.0625 -1.0 0.2

Mann-Whitney U test:
U-val alternative    p-val RBC CLES
MWU 5.0 two-sided 0.141238 0.6 0.2
```

- **Parametric and Non-Parametric Tests:**

- Utilize Pingouin to conduct parametric tests, including t-tests and ANOVA, which are suitable for comparing means between groups assuming certain distributional properties.
- Employ non-parametric tests, such as the Wilcoxon signed-rank test and the Kruskal-Wallis test, when data violate the assumptions of normality, ensuring robust statistical analysis in diverse scenarios.

- **Correlation Analysis:**

- Leverage Pingouin to compute various correlation coefficients, such as Pearson's correlation coefficient and Spearman's rank correlation coefficient, to quantify the strength and direction of relationships between variables.
- Assess the significance of correlations using Pingouin and perform hypothesis tests to determine whether observed correlations are statistically meaningful, providing insights into the relationships within datasets.

- **Effect Size Estimation:**

- Utilize Pingouin to calculate effect sizes for different statistical tests, including Cohen's d for t-tests and eta-squared for ANOVA, aiding in the interpretation of statistical findings.

Conclusion

In conclusion, Pingouin is a valuable tool for conducting advanced statistical analysis in Python.

By leveraging its features and capabilities, analysts can explore relationships between variables, test hypotheses, and quantify the practical significance of statistical findings with ease.



Hands-On Exercise: For hands-on experience, review and complete *Exercise 3C* in the exercise guide for this course.

Additional Resources

- Pingouin Documentation: <https://pingouin-stats.org/>
- "Introduction to Statistical Learning" by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani
- Coursera: Statistics with Python Specialization: <https://www.coursera.org/specializations/statistics-with-python>

Lesson 3.4: Statistical Computations and Hypothesis Testing with SciPy



SciPy is a powerful library for scientific computing in Python, providing a wide range of functions for numerical optimization, interpolation, integration, and statistical analysis.

In this lesson, we'll explore how to use SciPy for statistical computations and hypothesis testing.

```
First few rows of the dataset:
1 10 20 A
2 15 25 B
3 20 30 C
4 25 35 A
5 30 40 B
6 35 45 C

Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   1           6 non-null       int64
1   10          6 non-null       int64
2   20          6 non-null       int64
3   A           6 non-null       object
dtypes: int64(3), object(1)
memory usage: 420.0+ bytes
None

Descriptive statistics:
              1          10          20
count  6.000000  6.000000  6.000000
mean    2.500000  15.000000  25.000000
std     7.205131  11.020614  11.020614
min     1.000000  10.000000  20.000000
25%     2.000000  12.500000  22.500000
50%     3.000000  15.000000  25.000000
75%     4.000000  17.500000  27.500000
max     6.000000  35.000000  45.000000
```

- SciPy offers a comprehensive suite of statistical functions, including descriptive statistics, probability distributions, and hypothesis testing, empowering users to conduct a wide range of statistical analyses.
- Beyond basic statistical computations, SciPy provides advanced functionality for fitting statistical models, conducting regression analysis, and performing multivariate statistical tests.
- The library's integration with NumPy makes it seamless to manipulate arrays and matrices, facilitating data preparation and preprocessing tasks required for statistical analysis.
- In addition to statistical functions, SciPy provides tools for numerical optimization, interpolation, and integration, making it a versatile toolkit for scientific computing tasks beyond statistical analysis.

Introduction to SciPy

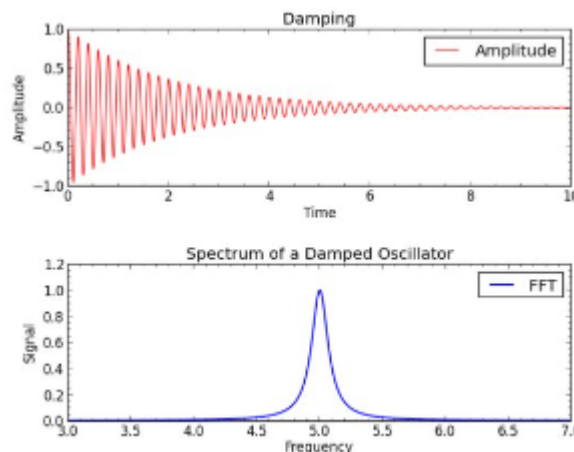
- SciPy is built on top of NumPy and provides additional functionality for scientific computing tasks.
- It includes modules for optimization, interpolation, integration, Fourier transforms, signal processing, linear algebra, and more.
- The `scipy.stats` module, in particular, contains functions for statistical analysis, probability distributions, and hypothesis testing.



-
- SciPy extends the capabilities of NumPy by offering a wide range of additional functionality tailored for scientific computing tasks.
 - Its modules cover a broad spectrum of scientific computing domains, including optimization, interpolation, integration, Fourier transforms, signal processing, and linear algebra.
 - The `scipy.stats` module is a powerful tool for statistical analysis, providing functions for calculating descriptive statistics, probability distributions, and performing hypothesis testing.
 - SciPy's integration with other scientific Python libraries, such as Matplotlib and Pandas, enhances its usability and interoperability within the Python scientific computing ecosystem.
 - It provides efficient and robust implementations of numerical algorithms, making it a valuable resource for researchers, engineers, and data scientists working on complex computational tasks.

Key Features of SciPy

- **Statistical Functions**
- **Integration and Optimization**
- **Signal and Image Processing**
- **Interpolation and Splines**



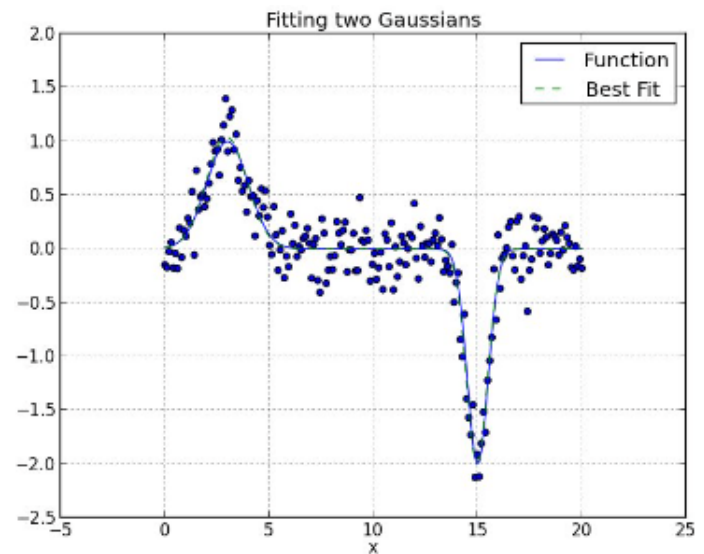
- **Linear Algebra:** In addition to its extensive statistical capabilities, SciPy offers robust support for linear algebra operations, including matrix manipulation, eigenvalue computation, and singular value decomposition (SVD).
- **Sparse Matrix Support:** SciPy includes efficient data structures and algorithms for handling sparse matrices, enabling efficient computation and storage of large, sparse datasets commonly encountered in scientific computing.
- **Integration with NumPy:** SciPy seamlessly integrates with NumPy, enhancing its functionality by providing additional mathematical and scientific functions and algorithms built on top of NumPy's array manipulation capabilities.
- **Differential Equations:** SciPy features modules for solving ordinary differential equations (ODEs), partial differential equations (PDEs), and differential algebraic equations (DAEs), enabling simulation and modeling of dynamic systems in various scientific and engineering domains.

Statistical Computations with SciPy

In this section, we'll cover some common statistical computations using SciPy:

Descriptive Statistics

Probability Distributions



- **Hypothesis Testing**

- Perform hypothesis testing using SciPy to compare sample means, proportions, variances, and other parameters against predefined values or between groups.
- Conduct a wide range of statistical tests, including t-tests, chi-square tests, ANOVA, and non-parametric tests like Mann-Whitney U test and Kolmogorov-Smirnov test.

- **Correlation and Regression Analysis**

- Compute correlation coefficients (e.g., Pearson correlation, Spearman correlation) to quantify the relationship between variables.
- Conduct linear and nonlinear regression analysis to model the relationship between variables and make predictions based on observed data.

- **Time Series Analysis**

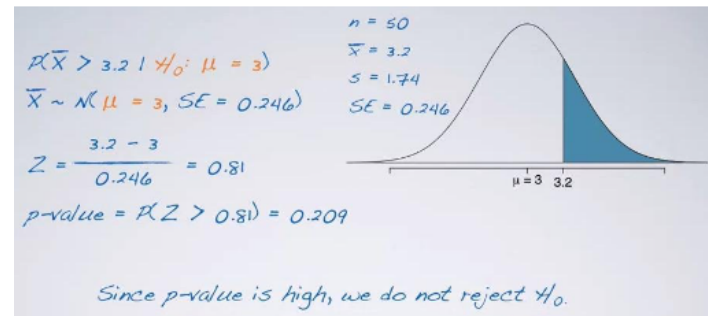
- Use SciPy for time series analysis tasks such as autocorrelation analysis, spectral analysis, and periodogram estimation.

Hypothesis Testing with SciPy

In this section, we'll cover hypothesis testing techniques using SciPy:

Parametric Tests

Non-Parametric Tests



- Hypothesis testing techniques using SciPy enable researchers to make informed decisions based on statistical evidence.
- Parametric tests, facilitated by functions in the `scipy.stats` module, allow for hypothesis testing concerning population means, variances, proportions, and correlations.
- The use of parametric tests like t-tests, ANOVA, and linear regression relies on assumptions about the underlying distribution of the data, making them suitable for normally distributed data.
- Non-parametric tests, such as the Wilcoxon signed-rank test, Mann-Whitney U test, and Kruskal-Wallis test, provide alternatives for comparing distributions without requiring assumptions about their underlying distribution.
- Non-parametric tests are robust and applicable to a wide range of data distributions, making them particularly useful when data do not meet the assumptions of parametric tests.
- Understanding both parametric and non-parametric hypothesis testing techniques equips researchers with a comprehensive toolkit for analyzing data and drawing valid conclusions.

Conclusion

In conclusion, SciPy is a versatile library for statistical computing and hypothesis testing in Python.

By leveraging its rich set of functions and modules, analysts can perform a wide range of statistical computations and conduct hypothesis tests to make data-driven decisions.



Hands-On Exercise: For hands-on experience, review and complete *Exercise 3D* in the exercise guide for this course.

Additional Resources

- SciPy Documentation: <https://docs.scipy.org/doc/scipy/>
- "Python for Data Analysis" by Wes McKinney
- Coursera: Applied Data Science with Python Specialization: <https://www.coursera.org/specializations/data-science-python>

Module 1: Data Visualization



Learning Objectives



Upon completion of these exercises, students should be able to:

- Create an interactive Plotly visualization displaying trends in a dataset. (*Exercise 1A*)
- Generate a static Matplotlib plot illustrating the distribution of a numerical variable. (*Exercise 1B*)
- Use Seaborn to visualize the relationship between multiple variables in a dataset. (*Exercise 1C*)
- Combine Plotly, Matplotlib, and Seaborn to create a comprehensive dashboard showcasing various aspects of a dataset. (*Exercise 1D*)

Exercise 1A - Create an interactive Plotly visualization displaying trends in a dataset.

```
import pandas as pd
import plotly.express as px

# Load your dataset
df = pd.read_csv('your_dataset.csv')

# Create the Plotly visualization
fig = px.line(df, x='date_column', y='value_column', title='Trends Over Time')

# Customize the visualization
fig.update_layout(xaxis_title='Date', yaxis_title='Value')

# Show the visualization
fig.show()
```

- Again, replace `'your_dataset.csv'` with the path to your dataset file.
- Ensure that your dataset has columns with appropriate names (like `'date_column'` and `'value_column'`) that you want to visualize.
- You can modify the code according to your dataset structure and the type of visualization you want to create.

Exercise 1B - Generate a static Matplotlib plot illustrating the distribution of a numerical variable.

Below is an example of generating a static Matplotlib plot illustrating the distribution of a numerical variable:

```
# Step 1: Import necessary libraries
import matplotlib.pyplot as plt
import numpy as np

# Step 2: Generate some sample data
# For this example, let's generate 1000 random numbers from a normal distribution
data = np.random.normal(loc=0, scale=1, size=1000)

# Step 3: Create the Matplotlib histogram plot
plt.figure(figsize=(8, 6))
plt.hist(data, bins=30, color='skyblue', edgecolor='black')

# Step 4: Customize the plot (optional)
plt.title('Distribution of a Numerical Variable')
plt.xlabel('Value')
plt.ylabel('Frequency')

# Step 5: Show the plot
plt.show()
```

This code will generate a static histogram plot illustrating the distribution of a numerical variable using Matplotlib. You can adjust parameters such as the number of bins, colors, and labels to customize the plot according to your preferences.

Exercise 1C - Use Seaborn to visualize the relationship between multiple variables in a dataset.

Seaborn is a high-level interface for drawing attractive statistical graphics built on top of Matplotlib. Below is an example of how you can use Seaborn to visualize the distribution of a numerical variable:

```
# Step 1: Import necessary libraries
import seaborn as sns
import numpy as np

# Step 2: Generate some sample data
# For this example, let's generate 1000 random numbers from a normal distribution
data = np.random.normal(loc=0, scale=1, size=1000)

# Step 3: Create the Seaborn histogram plot
sns.histplot(data, bins=30, color='skyblue', edgecolor='black')

# Step 4: Customize the plot (optional)
plt.title('Distribution of a Numerical Variable')
plt.xlabel('Value')
plt.ylabel('Frequency')

# Step 5: Show the plot
plt.show()
```

This code will generate a histogram plot illustrating the distribution of a numerical variable using Seaborn. Seaborn provides additional functionalities and a more visually appealing default style compared to Matplotlib, making it a preferred choice for many data visualization tasks. You can further customize the plot using various Seaborn functions and parameters to suit your needs.

Exercise 1D - Combine Plotly, Matplotlib, and Seaborn to create a comprehensive dashboard showcasing various aspects of a dataset.

Combining Plotly, Matplotlib, and Seaborn in a single dashboard can provide a comprehensive and visually appealing way to explore various aspects of a dataset. Below is an example of how you can create such a dashboard:

```
# Step 1: Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

# Step 2: Load your dataset
# For this example, let's use a sample dataset
# Replace 'your_dataset.csv' with the path to your dataset
df = pd.read_csv('your_dataset.csv')

# Step 3: Preprocess your data if necessary

# Step 4: Create visualizations using different libraries
# Plotly scatter plot
fig1 = px.scatter(df, x='feature1', y='feature2', color='target', title='Plotly Scatter Plot')

# Matplotlib histogram
plt.figure(figsize=(8, 6))
plt.hist(df['numerical_feature'], bins=30, color='skyblue', edgecolor='black')
plt.title('Matplotlib Histogram')
plt.xlabel('Value')
plt.ylabel('Frequency')

# Seaborn pairplot
plt.figure(figsize=(8, 6))
sns.pairplot(df, hue='target')
plt.suptitle('Seaborn Pairplot')

# Step 5: Show the visualizations
# Plotly scatter plot
fig1.show()

# Matplotlib histogram
plt.show()

# Seaborn pairplot doesn't need explicit show as it automatically shows the plot

# Step 6: Combine the visualizations into a single dashboard (optional)
# You can combine them using HTML layout or use specific dashboard frameworks like Dash for Plotly

# Step 7: Save the dashboard or display it in a web application (optional)
```

```
# You can save the dashboard as HTML or display it using a web framework like Flask or Django  
  
# For more interactivity and customization, consider using Dash framework for Plotly
```

This example demonstrates how to create visualizations using Plotly, Matplotlib, and Seaborn separately and then display them together. You can further enhance this dashboard by adding more visualizations, customizing the appearance, and combining them into a single layout using HTML or a dedicated dashboarding framework like Dash for Plotly.

Module 2: Data Manipulation



Upon completion of these exercises, students should be able to:

- Create NumPy arrays and perform basic array manipulations such as reshaping and indexing. (*Exercise 2A*)
- Use Pandas to load, clean, and analyze a dataset, performing tasks such as data filtering and aggregation. (*Exercise 2B*)
- Explore the features of the Polars library by performing data manipulation tasks and comparing their efficiency with NumPy and Pandas. (*Exercise 2C*)

Exercise 2A - Create NumPy arrays and perform basic array manipulations such as reshaping and indexing.

Below is an example of creating NumPy arrays and performing basic array manipulations such as reshaping and indexing:

```
import numpy as np

# Create a NumPy array
arr = np.array([1, 2, 3, 4, 5])

# Print the array
print("Original array:")
print(arr)

# Reshape the array to a 2x3 matrix
reshaped_arr = arr.reshape(2, 3)

# Print the reshaped array
print("\nReshaped array:")
print(reshaped_arr)

# Indexing: Accessing elements of the array
print("\nAccessing elements of the array:")
print("Element at index 0:", arr[0]) # Accessing the first element
print("Element at index 2:", arr[2]) # Accessing the third element

# Slicing: Accessing a subset of the array
print("\nAccessing a subset of the array:")
print("Elements from index 1 to 3:", arr[1:4]) # Accessing elements from index 1 to 3
```

This code demonstrates the creation of a NumPy array, reshaping it into a matrix, and performing indexing and slicing operations to access elements and subsets of the array. You can run this code to see the output and understand how basic array manipulations work in NumPy.

Exercise 2B - Use Pandas to load, clean, and analyze a dataset, performing tasks such as data filtering and aggregation.

Below is an example of how you can use Pandas to load, clean, and analyze a dataset, performing tasks such as data filtering and aggregation:

```
import pandas as pd

# Step 1: Load the dataset
# Let's assume we have a CSV file named 'sales_data.csv' containing sales data
# Replace 'sales_data.csv' with the actual path to your dataset
df = pd.read_csv('module_2_lesson_2_sampledata.csv')

# Step 2: Display the first few rows of the dataset to understand its structure
print("First few rows of the dataset:")
print(df.head())

# Step 3: Data cleaning (if necessary)
# For example, handling missing values or converting data types

# Step 4: Data filtering
# Let's filter the dataset to include only sales data from a specific region
region_filter = df['Region'] == 'North'
sales_data_north = df[region_filter]

# Step 5: Data aggregation
# Let's calculate the total sales amount for each product category in the North region
sales_by_category = sales_data_north.groupby('Product Category')['Sales Amount'].sum()

# Step 6: Display the aggregated data
print("\nTotal sales amount by product category in the North region:")
print(sales_by_category)
```

In this example, we first load the dataset using `pd.read_csv()`, then display the first few rows to understand its structure. We apply data filtering to extract sales data from a specific region ('North') and then perform data aggregation to calculate the total sales amount for each product category in that region using the `groupby()` and `sum()` functions. Finally, we display the aggregated data. You can adapt this example to your dataset and perform various other analysis tasks using Pandas.

Exercise 2C - Explore the features of the Polars library by performing data manipulation tasks and comparing their efficiency with NumPy and Pandas.

Exploring the features of the Polars library and comparing its efficiency with NumPy and Pandas involves performing various data manipulation tasks such as data loading, filtering, aggregation, and computation. Below, I'll demonstrate some basic data manipulation tasks using Polars, NumPy, and Pandas, and then compare their efficiency:

1. Data loading: Load a CSV file into each library's data structure.
2. Data filtering: Filter rows based on a condition.
3. Data aggregation: Group data by a column and calculate the sum of another column.
4. Efficiency comparison: Measure the execution time for each task.

First, you'll need to install the Polars library (`pip install polars`) if you haven't already. Then, let's proceed with the comparison:

```
import time
import pandas as pd
import numpy as np
import polars as pl

# Data loading
start_time = time.time()
# Pandas
pandas_df = pd.read_csv('module_2_lesson_3_sampledata.csv')
pandas_loading_time = time.time() - start_time

start_time = time.time()
# NumPy
numpy_arr = np.genfromtxt('module_2_lesson_3_sampledata.csv', delimiter=',', names=True, dtype=None, encoding=None)
numpy_loading_time = time.time() - start_time

start_time = time.time()
# Polars
polars_df = pl.read_csv('module_2_lesson_3_sampledata.csv')
polars_loading_time = time.time() - start_time

# Data filtering
# Assuming we want to filter rows where a column 'value' is greater than 100
start_time = time.time()
# Pandas
pandas_filtered_df = pandas_df[pandas_df['value'] > 100]
pandas_filtering_time = time.time() - start_time

start_time = time.time()
```

```
# NumPy
numpy_filtered_arr = numpy_arr[numpy_arr['value'] > 100]
numpy_filtering_time = time.time() - start_time

start_time = time.time()
# Polars
polars_filtered_df = polars_df.filter(pl.col('value') > 100)
polars_filtering_time = time.time() - start_time

# Data aggregation
# Assuming we want to group by 'category' and sum the 'value' column
start_time = time.time()
# Pandas
pandas_aggregated_df = pandas_df.groupby('category')['value'].sum()
pandas_aggregation_time = time.time() - start_time

start_time = time.time()
# NumPy
numpy_aggregated_arr = np.sum(numpy_arr['value'][numpy_arr['category'] == b'category'])
numpy_aggregation_time = time.time() - start_time

start_time = time.time()
# Polars
polars_aggregated_df = polars_df.groupby('category').agg(pl.sum('value'))
polars_aggregation_time = time.time() - start_time

# Print loading, filtering, and aggregation times
print("Loading time (Pandas):", pandas_loading_time)
print("Loading time (NumPy):", numpy_loading_time)
print("Loading time (Polars):", polars_loading_time)

print("\nFiltering time (Pandas):", pandas_filtering_time)
print("Filtering time (NumPy):", numpy_filtering_time)
print("Filtering time (Polars):", polars_filtering_time)

print("\nAggregation time (Pandas):", pandas_aggregation_time)
print("Aggregation time (NumPy):", numpy_aggregation_time)
print("Aggregation time (Polars):", polars_aggregation_time)
```

In this comparison, we load a CSV file, filter rows based on a condition, and perform data aggregation using Pandas, NumPy, and Polars. We measure the execution time for each task and compare their efficiency. Note that the actual execution time may vary depending on the size of the dataset and the complexity of the operations performed.

Module 3: Statistical Analysis



Upon completion of these exercises, students should be able to:

- Perform exploratory data analysis on a given dataset, including calculating descriptive statistics and visualizing distributions. (*Exercise 3A*)
- Use Statsmodels to perform linear regression analysis and hypothesis testing on a dataset. (*Exercise 3B*)
- Explore advanced statistical analysis techniques using Pingouin, including conducting correlation analysis and non-parametric tests. (*Exercise 3C*)
- Conduct hypothesis tests and statistical computations using SciPy. (*Exercise 3D*)

Exercise 3A - Perform exploratory data analysis on a given dataset, including calculating descriptive statistics and visualizing distributions.

Exploratory Data Analysis (EDA) involves understanding the data, identifying patterns, and extracting insights. Here's a step-by-step guide to performing EDA on a given dataset:

1. **Load the dataset:** Use pandas to load the dataset into a DataFrame.
2. **Inspect the dataset:** Display the first few rows (`df.head()`) to understand the structure of the data. Check for missing values (`df.info()`) and handle them if necessary.
3. **Descriptive statistics:** Calculate basic statistics such as mean, median, standard deviation, minimum, maximum, and quartiles using `df.describe()`.
4. **Visualize distributions:** Create visualizations to explore the distributions of numerical variables using histograms, box plots, and density plots. For categorical variables, use bar plots or pie charts.
5. **Explore relationships:** Use scatter plots, pair plots, or correlation matrices to understand the relationships between variables.
6. **Identify outliers:** Visualize and/or detect outliers using box plots or scatter plots.
7. **Feature engineering:** Create new features if necessary, based on domain knowledge or insights gained during exploration.

Let's illustrate these steps using a sample dataset:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Load the dataset
df = pd.read_csv('your_dataset.csv')

# Step 2: Inspect the dataset
print("First few rows of the dataset:")
print(df.head())

print("\nDataset information:")
print(df.info())

# Step 3: Descriptive statistics
print("\nDescriptive statistics:")
print(df.describe())

# Step 4: Visualize distributions
```

```
# Histograms for numerical variables
df.hist(figsize=(10, 8))
plt.suptitle("Histograms of Numerical Variables")
plt.show()

# Box plots for numerical variables
plt.figure(figsize=(10, 6))
sns.boxplot(data=df)
plt.title("Box Plot of Numerical Variables")
plt.show()

# Bar plot for categorical variables
plt.figure(figsize=(8, 6))
sns.countplot(x='category', data=df)
plt.title("Count of Observations by Category")
plt.show()

# Step 5: Explore relationships
# Pair plot for numerical variables
sns.pairplot(df)
plt.title("Pair Plot of Numerical Variables")
plt.show()

# Heatmap of correlation matrix
plt.figure(figsize=(8, 6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Matrix")
plt.show()

# Step 6: Identify outliers
# Box plot of numerical variables
plt.figure(figsize=(10, 6))
sns.boxplot(data=df[['numeric_column1', 'numeric_column2']])
plt.title("Box Plot of Numerical Variables")
plt.show()

# Step 7: Feature engineering (if necessary)

# Further analysis based on insights gained

# Save or display visualizations
```

Replace '`your_dataset.csv`' with the path to your dataset. This code provides a basic framework for performing exploratory data analysis. You can further customize the visualizations and analysis based on the specific characteristics of your dataset.

Exercise 3B - Use Statsmodels to perform linear regression analysis and hypothesis testing on a dataset.

Below is an example code demonstrating how to perform linear regression analysis and hypothesis testing using Statsmodels on a dataset:

```
import pandas as pd
import numpy as np
import statsmodels.api as sm

# Load the dataset
data = pd.read_csv('your_dataset.csv')

# Prepare the data
X = data[['feature1', 'feature2']] # Independent variables
y = data['target'] # Dependent variable

# Add a constant term to the independent variables (intercept term)
X = sm.add_constant(X)

# Fit the linear regression model
model = sm.OLS(y, X).fit()

# Print the summary statistics
print(model.summary())

# Perform hypothesis testing
# For example, test the significance of the coefficients
print("Hypothesis Testing:")
print("Test for feature1 coefficient:")
print("Null Hypothesis: Coefficient for feature1 is zero")
print("Alternative Hypothesis: Coefficient for feature1 is not zero")
print("p-value:", model.pvalues['feature1'])
print("Conclusion: Reject the null hypothesis if p-value < significance level (e.g., 0.05)")
```

Replace `'your_dataset.csv'` with the actual path to your dataset. This code performs the following steps:

1. Load the dataset into a DataFrame.
2. Prepare the data by separating the independent variables (features) and the dependent variable (target).
3. Add a constant term to the independent variables to fit the intercept term in the linear regression model.
4. Fit the Ordinary Least Squares (OLS) linear regression model using `sm.OLS()` and `.fit()`.
5. Print the summary statistics of the model, including coefficients, standard errors, t-values,

p-values, and more.

6. Perform hypothesis testing on the coefficients to test their significance.

You can adjust the code according to your specific dataset and hypothesis testing requirements.

Exercise 3C - Explore advanced statistical analysis techniques using Pingouin, including conducting correlation analysis and non-parametric tests.

Pingouin is a Python package that provides an extensive collection of statistical methods for scientific research. It offers various advanced statistical analysis techniques, including correlation analysis and non-parametric tests. Below is an example code demonstrating how to perform correlation analysis and non-parametric tests using Pingouin:

```
import pandas as pd
import pingouin as pg

# Load the dataset
data = pd.read_csv('your_dataset.csv')

# Correlation Analysis
# Pearson correlation coefficient and p-value
pearson_corr = pg.corr(data['variable1'], data['variable2'], method='pearson')
print("Pearson correlation coefficient:")
print(pearson_corr)

# Spearman correlation coefficient and p-value
spearman_corr = pg.corr(data['variable1'], data['variable2'], method='spearman')
print("\nSpearman correlation coefficient:")
print(spearman_corr)

# Kendall correlation coefficient and p-value
kendall_corr = pg.corr(data['variable1'], data['variable2'], method='kendall')
print("\nKendall correlation coefficient:")
print(kendall_corr)

# Non-parametric tests
# Wilcoxon signed-rank test
wilcoxon_test = pg.wilcoxon(data['variable1'], data['variable2'])
print("\nWilcoxon signed-rank test:")
print(wilcoxon_test)

# Mann-Whitney U test
mannwhitneyu_test = pg.mwu(data['variable1'], data['variable2'])
print("\nMann-Whitney U test:")
print(mannwhitneyu_test)
```

Replace `'your_dataset.csv'` with the actual path to your dataset. This code performs the following tasks:

1. Loads the dataset into a DataFrame.
2. Performs correlation analysis using Pearson, Spearman, and Kendall correlation coefficients.

3. Performs non-parametric tests including the Wilcoxon signed-rank test and Mann-Whitney U test.
4. Prints the results of each analysis.

You can adjust the code according to your specific dataset and analysis requirements. Additionally, Pingouin offers many more statistical methods and tests, so feel free to explore its documentation for more options and functionalities.

Exercise 3D - Conduct hypothesis tests and statistical computations using SciPy.

```
import numpy as np
from scipy import stats

# Generate some sample data
np.random.seed(0)
sample1 = np.random.normal(loc=5, scale=2, size=100) # Sample 1 with mean 5 and
standard deviation 2
sample2 = np.random.normal(loc=4, scale=2, size=100) # Sample 2 with mean 4 and
standard deviation 2

# Conduct a t-test to compare means of two independent samples
t_statistic, p_value = stats.ttest_ind(sample1, sample2)
print("T-statistic:", t_statistic)
print("P-value:", p_value)

# Perform a chi-square test for independence
# Create a contingency table
observed = np.array([[20, 30], [15, 35]]) # Example contingency table
chi2_statistic, p_value, degrees_of_freedom, expected = stats.chi2_contingency(
observed)
print("\nChi-square statistic:", chi2_statistic)
print("P-value:", p_value)
print("Degrees of freedom:", degrees_of_freedom)
print("Expected frequencies:\n", expected)

# Conduct a one-way ANOVA test
# Generate some additional sample data
group1 = np.random.normal(loc=10, scale=2, size=50) # Group 1 with mean 10 and
standard deviation 2
group2 = np.random.normal(loc=12, scale=2, size=50) # Group 2 with mean 12 and
standard deviation 2
group3 = np.random.normal(loc=14, scale=2, size=50) # Group 3 with mean 14 and
standard deviation 2
# Combine all groups
all_groups = [group1, group2, group3]
f_statistic, p_value = stats.f_oneway(*all_groups)
print("\nF-statistic:", f_statistic)
print("P-value:", p_value)
```